

Time Series Analysis Final Project

Christian Etherson

2026-05-08

Contents

1	Part I: Electricity Generation Data	1
1.1	Exploratory Data Analysis	1
1.2	Stationarity Check	7
1.3	Model Fitting	11
1.4	Diagnostic Checking	14
1.5	Forecasting	23
2	Part II: Cryptocurrency Return Data	26
2.1	Exploratory Data Analysis	26
2.2	Arch Effect Test	29
2.3	Model Fitting	31
2.4	Diagnostic Checking	36
2.5	Volatility Interpretation	45
3	Sources	48

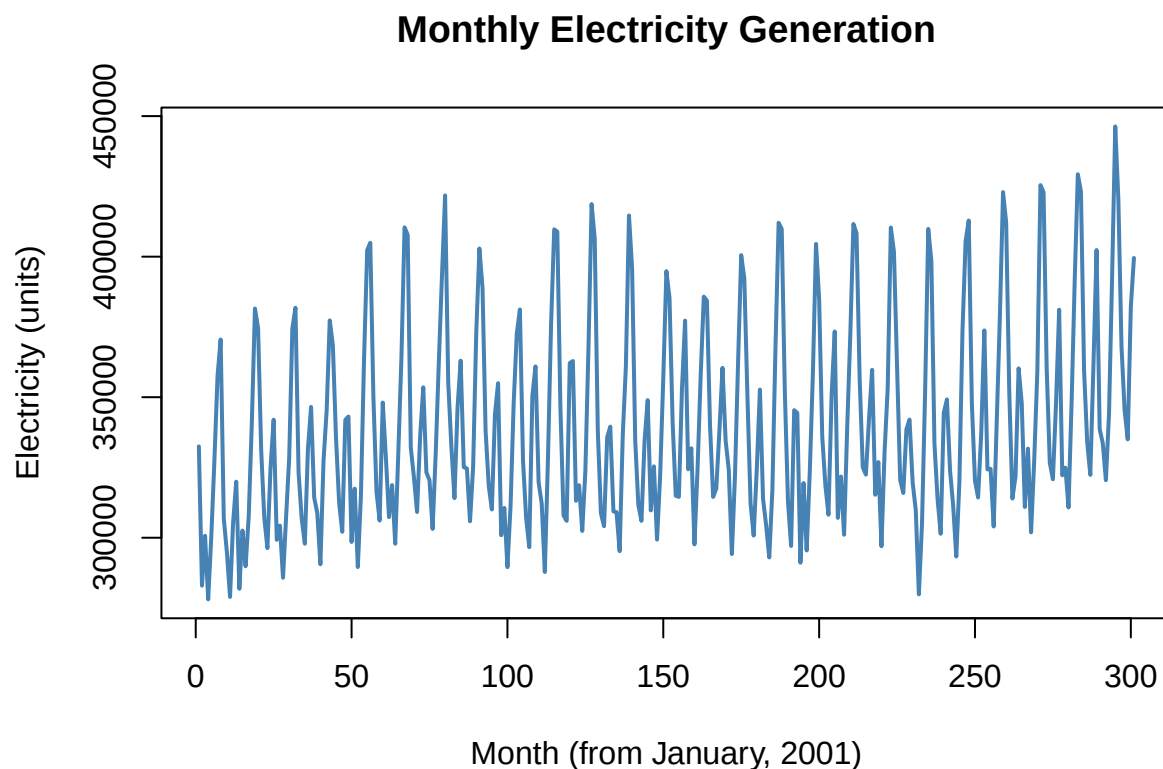
1 Part I: Electricity Generation Data

1.1 Exploratory Data Analysis

I will begin the examination of the monthly electricity generation data by plotting the time series, the autocorrelation function, and the partial autocorrelation

function. These graphs will give a good sense of the stationarity, seasonality, and trend of the time series as well as which models could or could not be a good fit.

```
library(readxl)
library(forecast)
monthly_generation <-
  read_excel("Net_generation_United_States_all_sectors_monthly.xlsx")
plot(monthly_generation$Electricity,
     type = "l",
     main = "Monthly Electricity Generation",
     xlab = "Month (from January, 2001)",
     ylab = "Electricity (units)",
     col = "steelblue",
     lwd = 2)
```



```

electricity_ts <- ts(monthly_generation$Electricity,
                     start = c(2001, 1))
electricity_ts_yearly <- ts(monthly_generation$Electricity,
                             start = c(2001, 1),
                             frequency = 12)

```

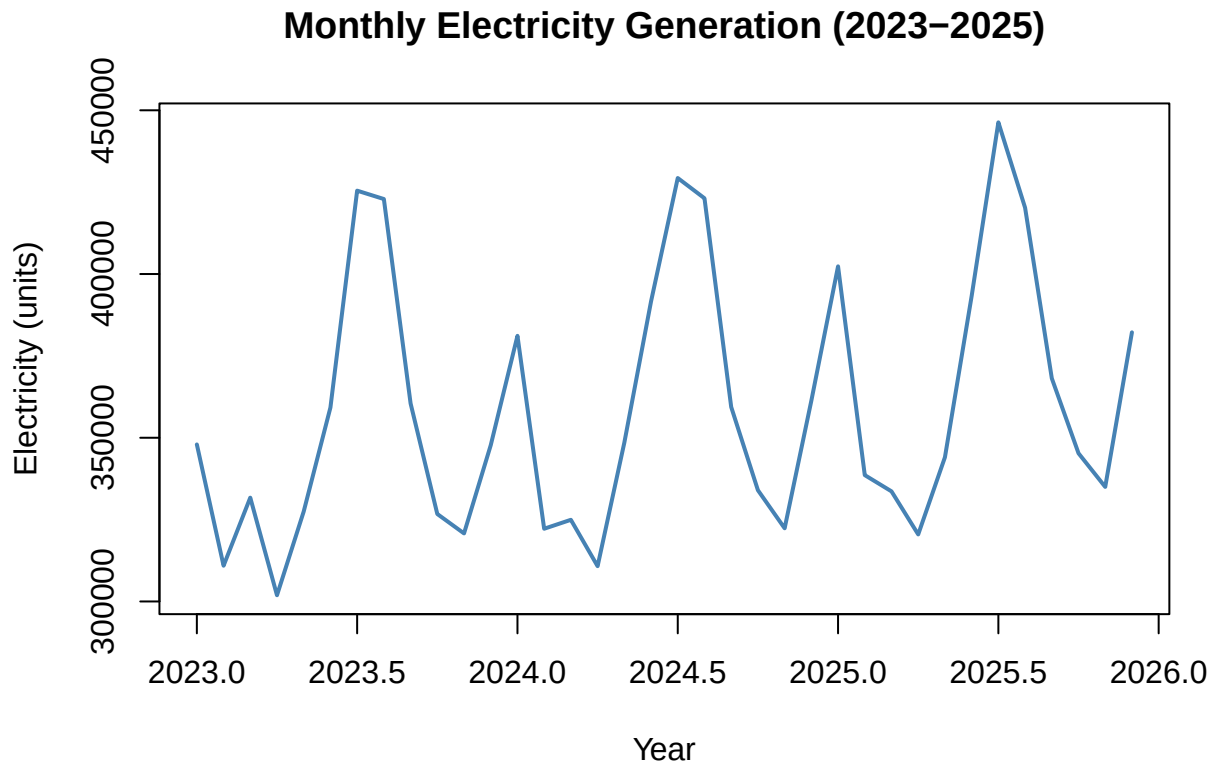
The time series graph of the monthly electricity generation in the United States appears seasonal with the electricity rising and falling in cyclical patterns. The trend appears to be increasing over time and the variance appears to be roughly constant. This makes sense intuitively because electricity demand is not constant throughout the year, and electricity demand generally rises over time. To accomodate this, electricity generation would also change throughout each year and rise over the years.

```

electricity_2yr <- window(electricity_ts_yearly, start = c(2023, 1), end = c(2025, 12))

plot(electricity_2yr,
     main = "Monthly Electricity Generation (2023-2025)",
     xlab = "Year",
     ylab = "Electricity (units)",
     type = "l",
     col = "steelblue",
     lwd = 2)

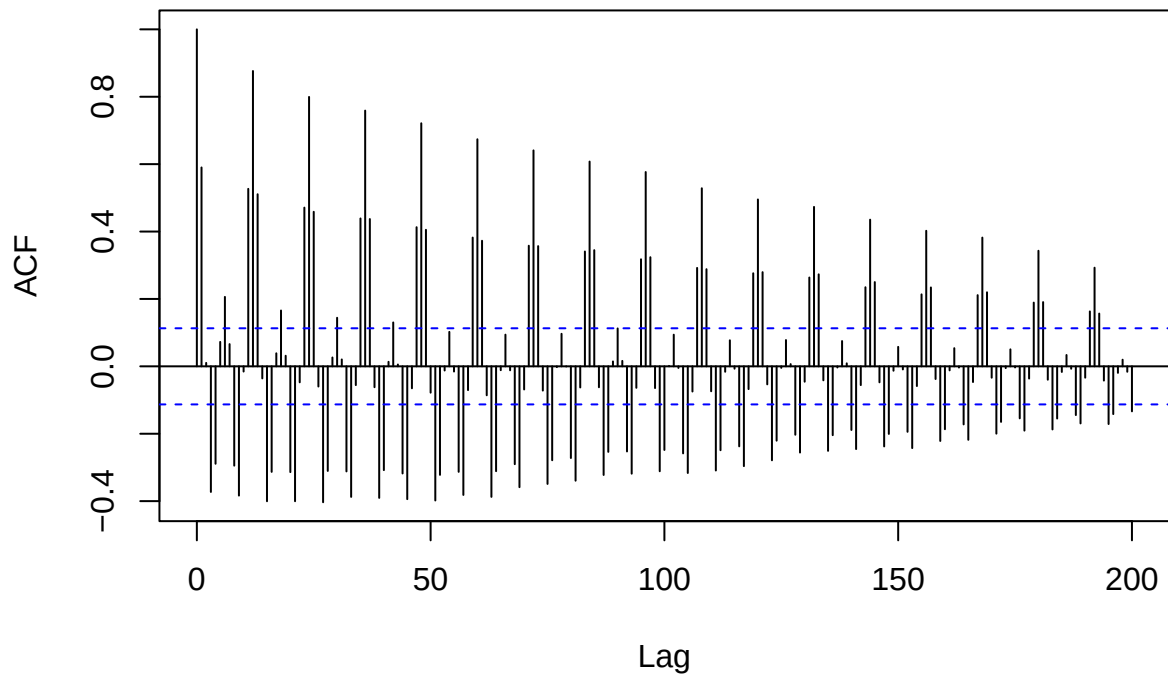
```



To examine the seasonality, I zoomed the graph in to 2023-2025. Across these years, it's clear that electricity generation peaks at the start of the year during the winter and again in the middle of the year during the summer. It is also notable that the winter peaks tend to be smaller than the summer peaks.

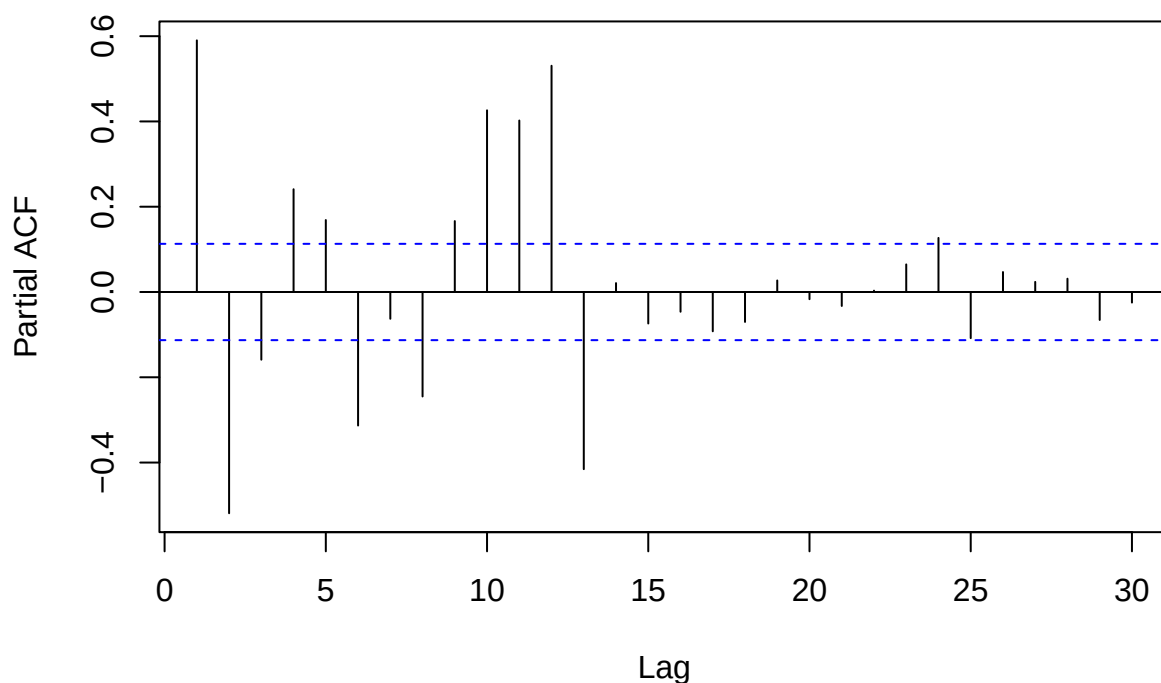
```
acf(electricity_ts, main = "ACF - Monthly Electricity Generation", lag.max = 200)
```

ACF – Monthly Electricity Generation



```
pacf(electricity_ts, main = "PACF – Monthly Electricity Generation", lag.max = 30)
```

PACF – Monthly Electricity Generation



The autocorrelation function and partial autocorrelation function graphs give insight into the structure of a time series by showing the correlations of the current term of the series and a term at a given lag. The autocorrelation function gives the correlation including indirect influence from intermediate lags, while the partial autocorrelation gives the direct correlation at each lag without any influence from intermediate lags. The autocorrelation function graph indicates nonstationarity and seasonality, as it converges very slowly over the lags and includes a cycle above and below the x-axis. The partial autocorrelation function graph indicates seasonality since the lags with the strongest correlation to a given term are the first, second, and lags surrounding the twelfth lag. In words, this indicates that a given months electricity generation is heavily explained by the generation from the two months before it and the generation from the same month one year before it. The pattern of significant spikes at early lags and lags surrounding lag 12 is consistent with a seasonal ARIMA model.

1.2 Stationarity Check

I will now try to quantify a lack of stationary and apply transformations to make the series stationary

```
library(tseries)
```

```
## Registered S3 method overwritten by 'quantmod':  
##   method      from  
##   as.zoo.data.frame zoo
```

```
adf.test(electricity_ts_yearly)
```

```
## Warning in adf.test(electricity_ts_yearly): p-value smaller than printed  
## p-value
```

```
##  
## Augmented Dickey-Fuller Test  
##  
## data: electricity_ts_yearly  
## Dickey-Fuller = -9.4476, Lag order = 6, p-value = 0.01  
## alternative hypothesis: stationary
```

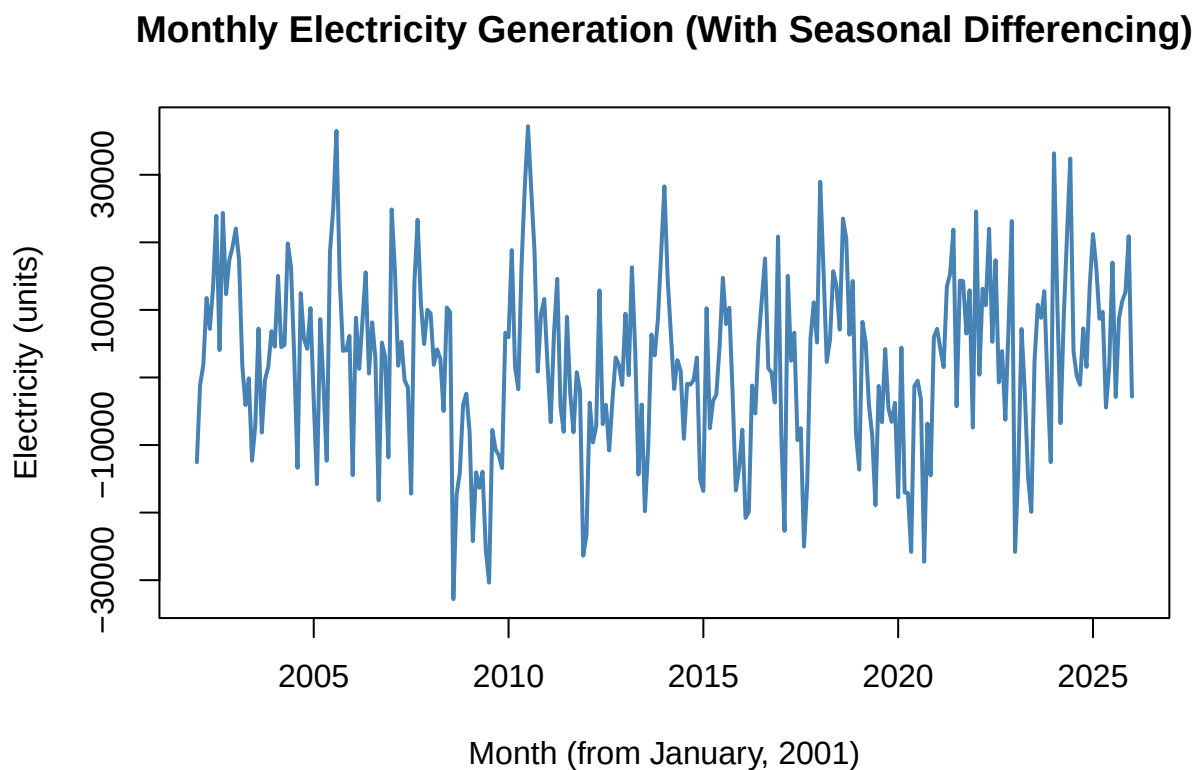
Interestingly, the ADF test shows a small p-value of 0.01, indicating that the series is stationary. While this indicates that the time series does not significantly trend upward as I theorized earlier, it does not capture the nonstationarity caused by the seasonal pattern revealed by the cyclic pattern of the ACF. Because of this, a regular differencing is not required, but a seasonal differencing will achieve full stationarity.

I will apply a difference at lag 12 to account for the yearly seasonal pattern.

```

electricity_seasonal_diff <- diff(electricity_ts_yearly, lag = 12)
plot(electricity_seasonal_diff,
     type = "l",
     main = "Monthly Electricity Generation (With Seasonal Differencing)",
     xlab = "Month (from January, 2001)",
     ylab = "Electricity (units)",
     col = "steelblue",
     lwd = 2)

```



The updated time series graph after applying the seasonal differencing now appears stationary. The variance appears constant, there is no upward or downward trend, and the yearly cyclic pattern is gone.

```
adf.test(electricity_seasonal_diff)
```

```
## Warning in adf.test(electricity_seasonal_diff): p-value smaller than printed
## p-value
```

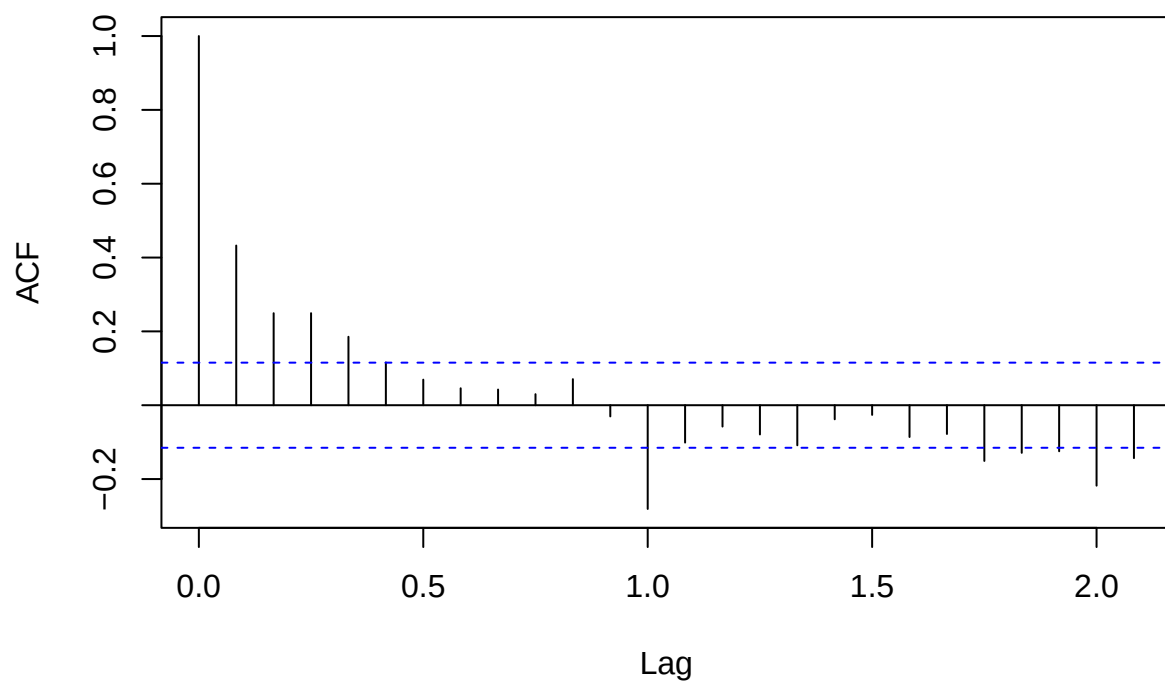


```
##
## Augmented Dickey-Fuller Test
##
## data: electricity_seasonal_diff
## Dickey-Fuller = -5.1011, Lag order = 6, p-value = 0.01
## alternative hypothesis: stationary
```

The ADF test for the differenced model also returns a small p-value of 0.01, which confirms the stationarity of the time series.

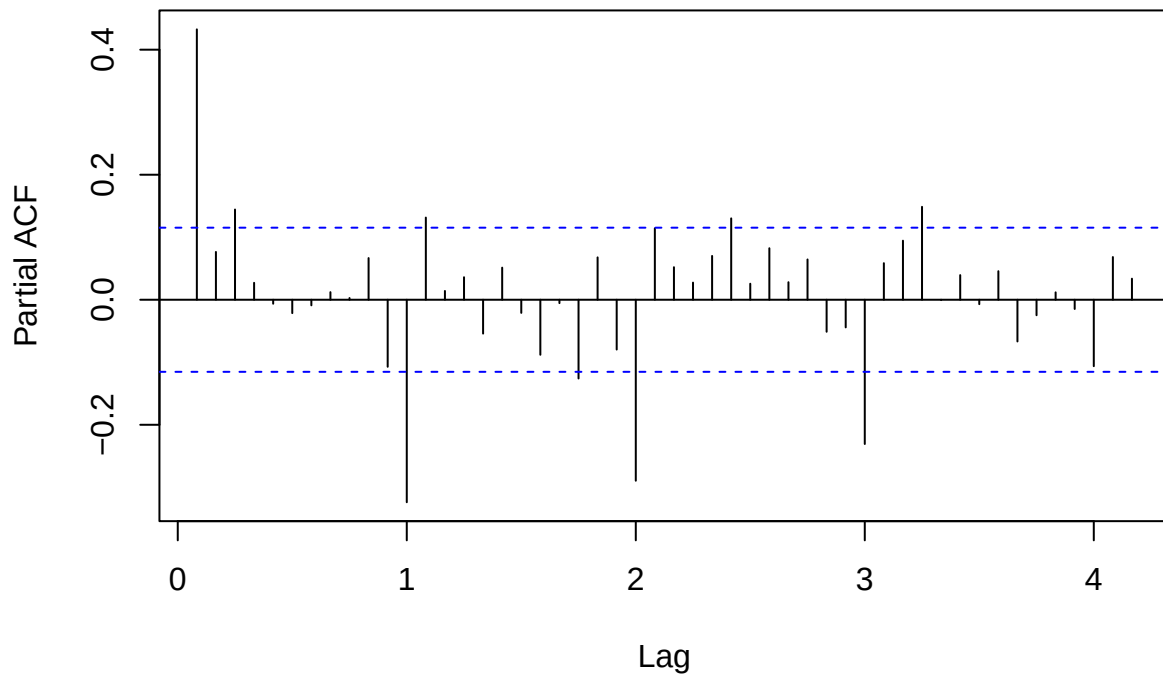
```
acf(electricity_seasonal_diff, main = "ACF - Monthly Electricity Generation (With Seasonal Differencing)")
```

ACF – Monthly Electricity Generation (With Seasonal Differencing)



```
pacf(electricity_seasonal_diff, main = "PACF - Monthly Electricity Generation (With Seasonal Differencing)")
```

PACF – Monthly Electricity Generation (With Seasonal Differencing)



The ACF graph has been changed greatly by the differencing and also indicates no seasonal nonstationarity. It has a large cutoff after the first lag, a spike at the twelfth lag, and a smaller spike at the twenty fourth lag. While the second and third lags are outside of the critical boundaries, they may not be practically significant, so this graph best shows a seasonal MA(1) model with a regular MA(1) component.

The PACF generally shows a trailing off of the significant autocorrelations which also indicates an MA component. However, there is an especially large spike at lag 1, so an AR(1) component should also be explored.

Finally, the variance appeared constant throughout the differenced data, so a log transformation was unnecessary.

1.3 Model Fitting

Based on the previous sections findings, I will consider four seasonal ARIMA models:

ARIMA(0,0,1)x(0,1,1)12: Containing a regular MA(1) component, a seasonal MA(1) component, and 1 seasonal differencing.

ARIMA(1,0,1)x(0,1,1)12: Containing a regular MA(1) component, a regular AR(1) component, a seasonal MA(1) component, and 1 seasonal differencing.

ARIMA(0,0,1)x(0,1,2)12: Containing a regular MA(1) component, a seasonal MA(2) component, and 1 seasonal differencing.

ARIMA(1,0,1)x(0,1,2)12: Containing a regular MA(1) component, a regular AR(1) component, a seasonal MA(2) component, and 1 seasonal differencing.

For each model, I will report the form of the model, it's coefficients, and it's AIC.

ARIMA(0,0,1)x(0,1,1)12

```
model1 <- arima(electricity_ts_yearly,
  order = c(0,0,1),
  seasonal = list(order = c(0,1,1), period = 12))
model1

##
## Call:
## arima(x = electricity_ts_yearly, order = c(0, 0, 1), seasonal = list(order = c(0,
##    1, 1), period = 12))
##
## Coefficients:
##          ma1      sma1
##      0.4532  -0.5819
## s.e.  0.0458   0.0639
##
## sigma^2 estimated as 114533447:  log likelihood = -3094.07,  aic = 6194.13
```

Model Form:

$$\nabla_{12}Y_t = \varepsilon_t + 0.4532\varepsilon_{t-1} - 0.5819\varepsilon_{t-12}$$

Estimated Coefficients: ma1: 0.4532, seasonal ma1: -0.5819

AIC: 6194.13

1.3.1 ARIMA(1,0,1)x(0,1,1)12

```
model2 <- arima(electricity_ts_yearly,
                 order = c(1,0,1),
                 seasonal = list(order = c(0,1,1), period = 12))
model2
```

```
##
```

```
## Call:
```

```
## arima(x = electricity_ts_yearly, order = c(1, 0, 1), seasonal = list(order = c(0,
##    1, 1), period = 12))
```

```
##
```

```
## Coefficients:
```

```
##          ar1          ma1          sma1
```

```
##          0.9440   -0.5580   -0.8418
```

```
## s.e.    0.0393    0.0993    0.0423
```

```
##
```

```
## sigma^2 estimated as 87462528:  log likelihood = -3059.9,  aic = 6127.8
```

Model Form:

$$\nabla_{12}Y_t = 0.9440Y_{t-1} + \varepsilon_t - 0.5580\varepsilon_{t-1} - 0.8418\varepsilon_{t-12}$$

Estimated Coefficients: ar1: 0.9440, ma1: -0.5580, seasonal ma1: -0.8418

AIC: 6127.8

1.3.2 ARIMA(0,0,1)x(0,1,2)12

```
model3 <- arima(electricity_ts_yearly,
                order = c(0,0,1),
                seasonal = list(order = c(0,1,2), period = 12))
model3

##
## Call:
## arima(x = electricity_ts_yearly, order = c(0, 0, 1), seasonal = list(order = c(0,
##    1, 2), period = 12))
##
## Coefficients:
##          ma1      sma1      sma2
##      0.4549 -0.4920 -0.1596
## s.e.  0.0459  0.0661  0.0642
##
## sigma^2 estimated as 1.12e+08:  log likelihood = -3091.14,  aic = 6190.28
```

Model Form:

$$\nabla_{12}Y_t = \varepsilon_t + 0.4549\varepsilon_{t-1} - 0.4920\varepsilon_{t-12} - 0.1596\varepsilon_{t-24}$$

Estimated Coefficients: ma1: 0.4549, seasonal ma1: -0.4920, seasonal ma2: -0.1596

AIC: 6190.28

1.3.3 ARIMA(1,0,1)x(0,1,2)12

```
model4 <- arima(electricity_ts_yearly,
                order = c(1,0,1),
                seasonal = list(order = c(0,1,2), period = 12))
model4
```

```
##
## Call:
## arima(x = electricity_ts_yearly, order = c(1, 0, 1), seasonal = list(order = c(0,
##      1, 2), period = 12))
##
## Coefficients:
##          ar1          ma1          sma1          sma2
##      0.9397 -0.5599 -0.7306 -0.1254
## s.e.  0.0400  0.0961  0.0706  0.0665
##
## sigma^2 estimated as 86436333:  log likelihood = -3058.21,  aic = 6126.42
```

$$\nabla_{12}Y_t = 0.9397Y_{t-1} + \varepsilon_t - 0.5599\varepsilon_{t-1} - 0.7306\varepsilon_{t-12} - 0.1254\varepsilon_{t-24}$$

Estimated Coefficients: ar1: 0.9397, ma1: -0.5599, seasonal ma1: -0.7306, seasonal ma2: -0.1254

AIC: 6126.42

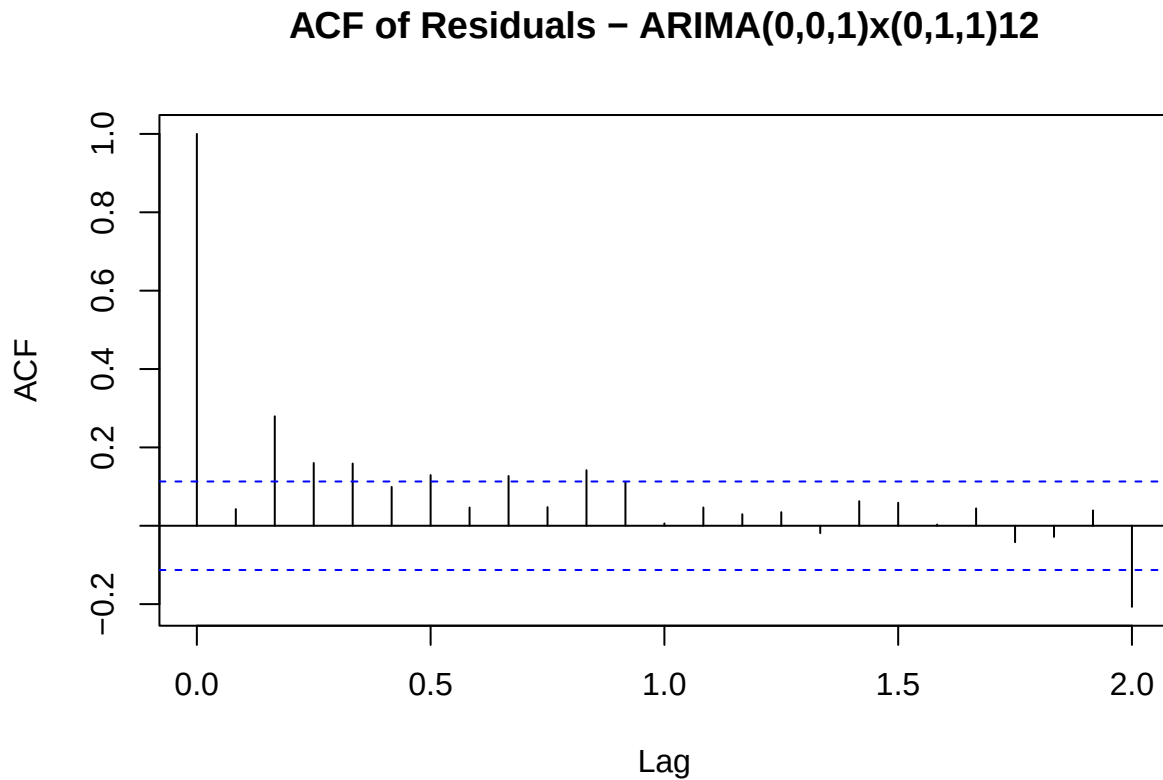
Model	AIC
$ARIMA(1, 0, 1) \times (0, 1, 2)_{12}$	6126.42
$ARIMA(1, 0, 1) \times (0, 1, 1)_{12}$	6127.80
$ARIMA(0, 0, 1) \times (0, 1, 2)_{12}$	6190.28
$ARIMA(0, 0, 1) \times (0, 1, 1)_{12}$	6194.13

1.4 Diagnostic Checking

For each model, I will examine the residual ACF, Ljung-Box test statistic, and normality of residuals to pick the best model out of the four. A well fitting model will show uncorrelated residuals tested visually by the residual ACF graph and quantitatively by the Ljung-Box test, and normally distributed residuals tested visually by the residual QQ plot and quantitatively by the Shapiro-Wilk test.

1.4.1 ARIMA(0,0,1)x(0,1,1)12

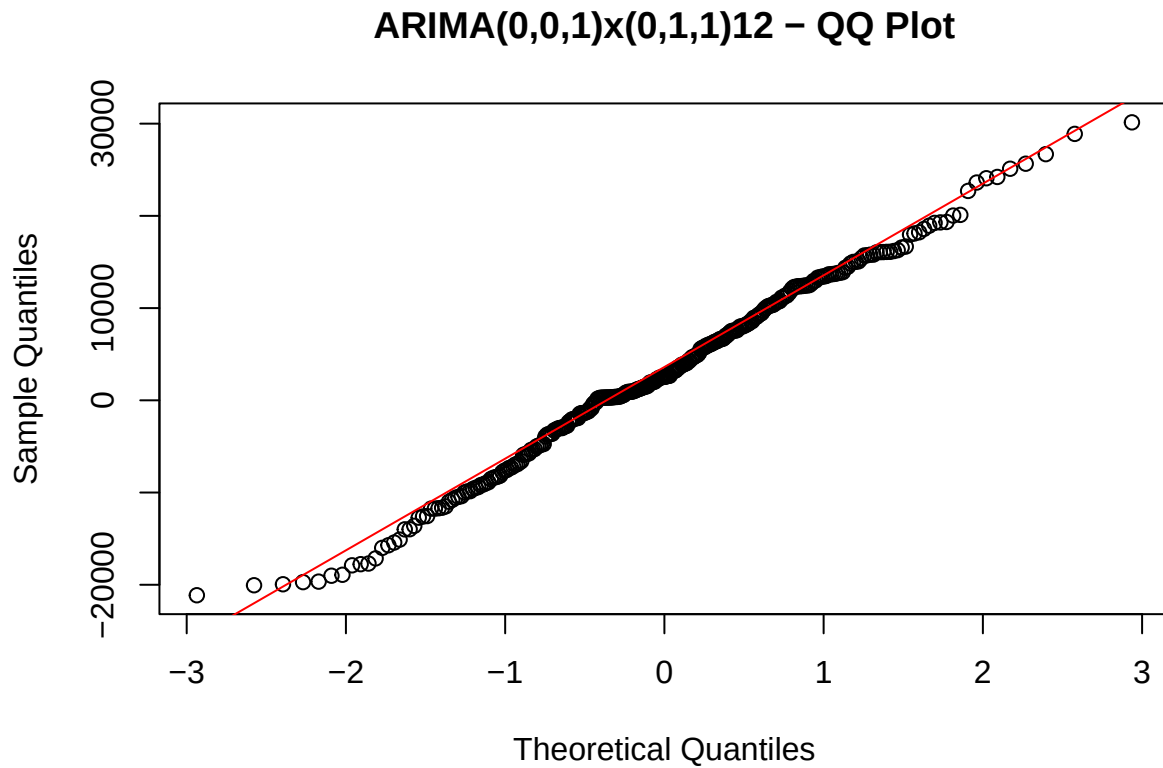
```
acf(residuals(model1), main = "ACF of Residuals - ARIMA(0,0,1)x(0,1,1)12")
```



```
Box.test(residuals(model1), lag = 20, type = "Ljung-Box")
```

```
##  
## Box-Ljung test  
##  
## data: residuals(model1)  
## X-squared = 69.246, df = 20, p-value = 2.417e-07
```

```
qqnorm(residuals(model1), main = "ARIMA(0,0,1)x(0,1,1)12 - QQ Plot");  
qqline(residuals(model1), col = "red")
```



```
shapiro.test(residuals(model1))
```

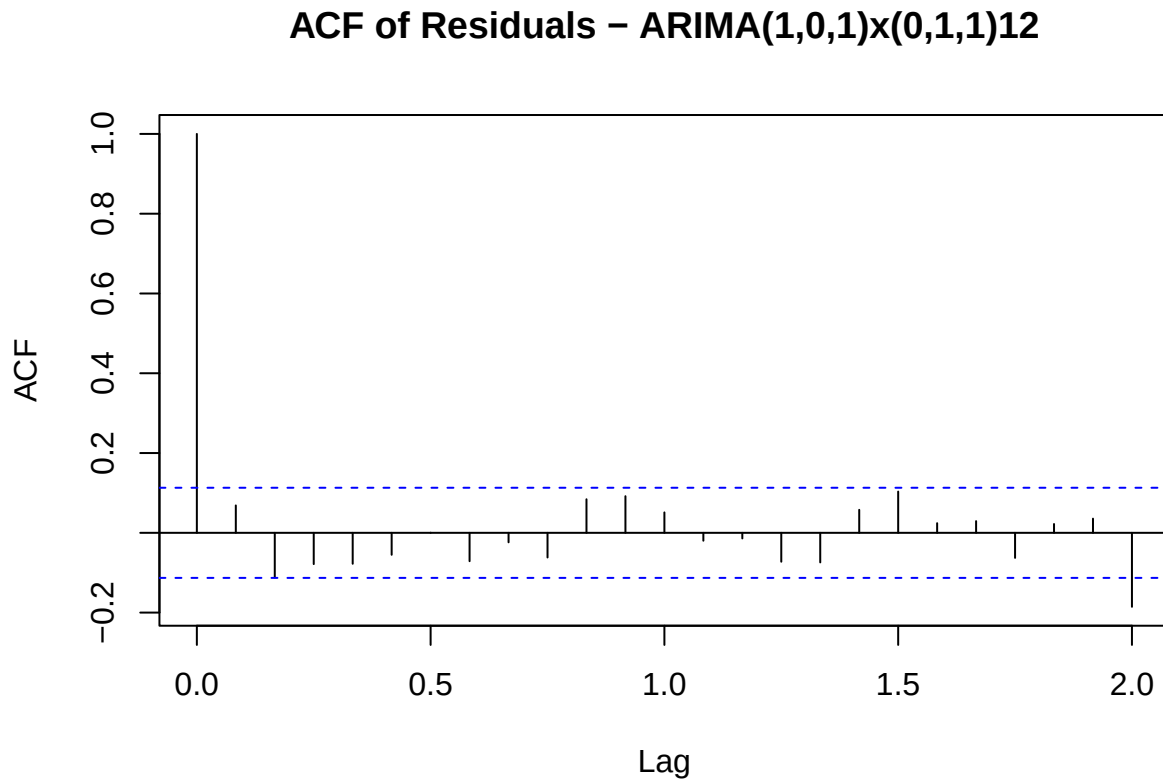
```
##  
##  Shapiro-Wilk normality test  
##  
## data:  residuals(model1)  
## W = 0.99385, p-value = 0.2611
```

Summary of Tests:

The ACF graph shows several lags breaking the critical boundaries, indicating the residuals are not independent. This result is confirmed by the Ljung-Box test which has a small p-value of $2.417e-07$, showing the residuals are significantly autocorrelated. The QQ plot shows the residuals are approximately normally distributed as the values generally remain close to the line. This result is confirmed by the Shapiro-Wilk test which has a large p-value of 0.2611, so I fail to reject the null hypothesis of normality.

1.4.2 ARIMA(1,0,1)x(0,1,1)12

```
acf(residuals(model2), main = "ACF of Residuals - ARIMA(1,0,1)x(0,1,1)12")
```

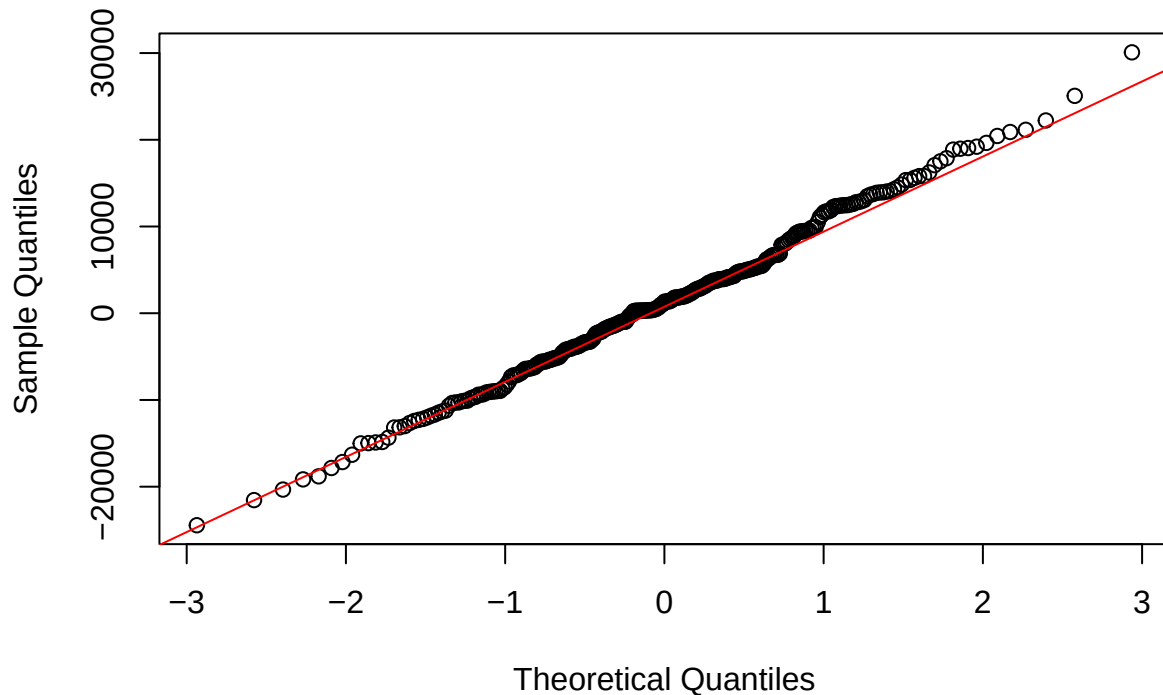


```
Box.test(residuals(model2), lag = 20, type = "Ljung-Box")
```

```
##  
## Box-Ljung test  
##  
## data: residuals(model2)  
## X-squared = 27.17, df = 20, p-value = 0.1306
```

```
qqnorm(residuals(model2), main = "ARIMA(1,0,1)x(0,1,1)12 - QQ Plot");  
qqline(residuals(model2), col = "red")
```

ARIMA(1,0,1)x(0,1,1)12 – QQ Plot



```
shapiro.test(residuals(model2))
```

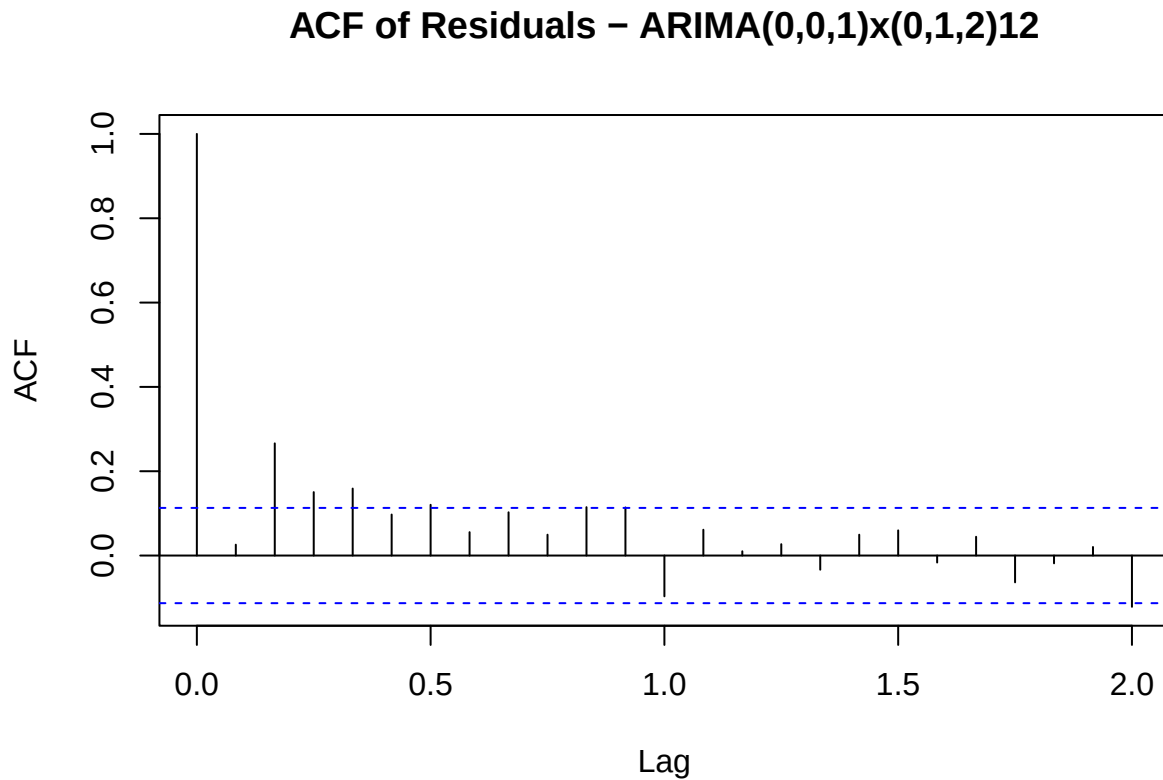
```
##  
##  Shapiro-Wilk normality test  
##  
## data:  residuals(model2)  
## W = 0.99653, p-value = 0.7578
```

Summary of tests:

The ACF graph shows only one lag slightly breaking the critical boundaries, indicating the residuals are independent. This result is confirmed by the Ljung-Box test which has a large p-value of 0.1306, so I fail to reject the null hypothesis that the residuals are not autocorrelated. The QQ plot shows the residuals are approximately normally distributed as the values generally remain close to the line. This result is confirmed by the Shapiro-Wilk test which has a large p-value of 0.7578, so I fail to reject the null hypothesis of normality.

1.4.3 ARIMA(0,0,1)x(0,1,2)12

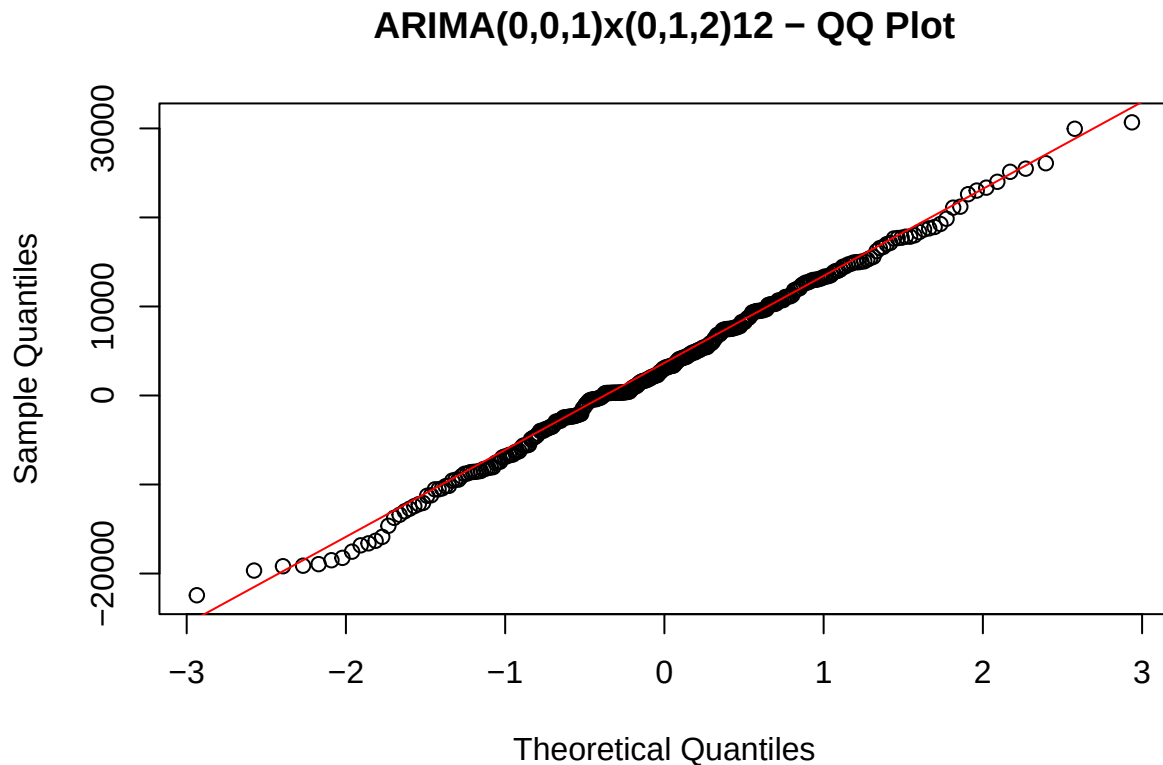
```
acf(residuals(model3), main = "ACF of Residuals - ARIMA(0,0,1)x(0,1,2)12")
```



```
Box.test(residuals(model3), lag = 20, type = "Ljung-Box")
```

```
##  
## Box-Ljung test  
##  
## data: residuals(model3)  
## X-squared = 64.521, df = 20, p-value = 1.391e-06
```

```
qqnorm(residuals(model3), main = "ARIMA(0,0,1)x(0,1,2)12 - QQ Plot"); qqline(residuals(model3))
```



```
shapiro.test(residuals(model3))
```

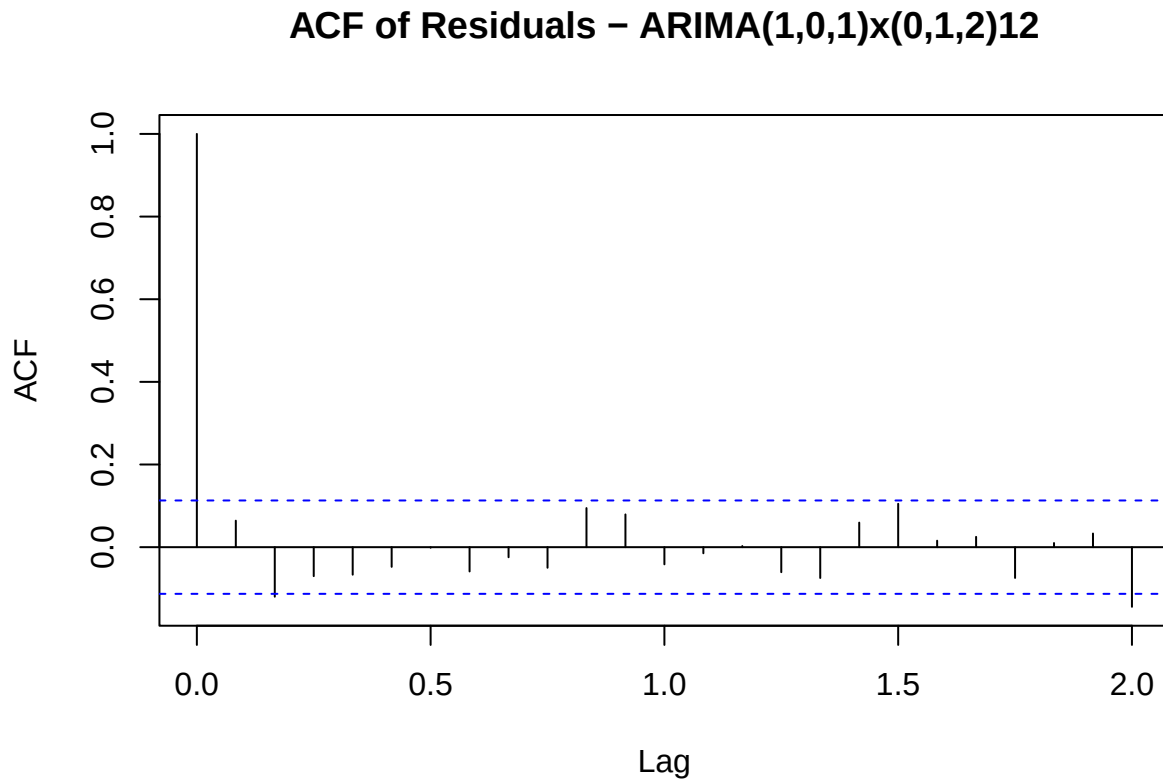
```
##  
##  Shapiro-Wilk normality test  
##  
## data:  residuals(model3)  
## W = 0.9966, p-value = 0.7743
```

Summary of tests:

The ACF graph shows several lags breaking the critical boundaries, indicating the residuals are not independent. This result is confirmed by the Ljung-Box test which has a small p-value of $1.391e-06$, showing the residuals are significantly autocorrelated. The QQ plot shows the residuals are approximately normally distributed as the values generally remain close to the line. This result is confirmed by the Shapiro-Wilk test which has a large p-value of 0.7743, so I fail to reject the null hypothesis of normality.

1.4.4 ARIMA(1,0,1)x(0,1,2)12

```
acf(residuals(model4), main = "ACF of Residuals - ARIMA(1,0,1)x(0,1,2)12")
```

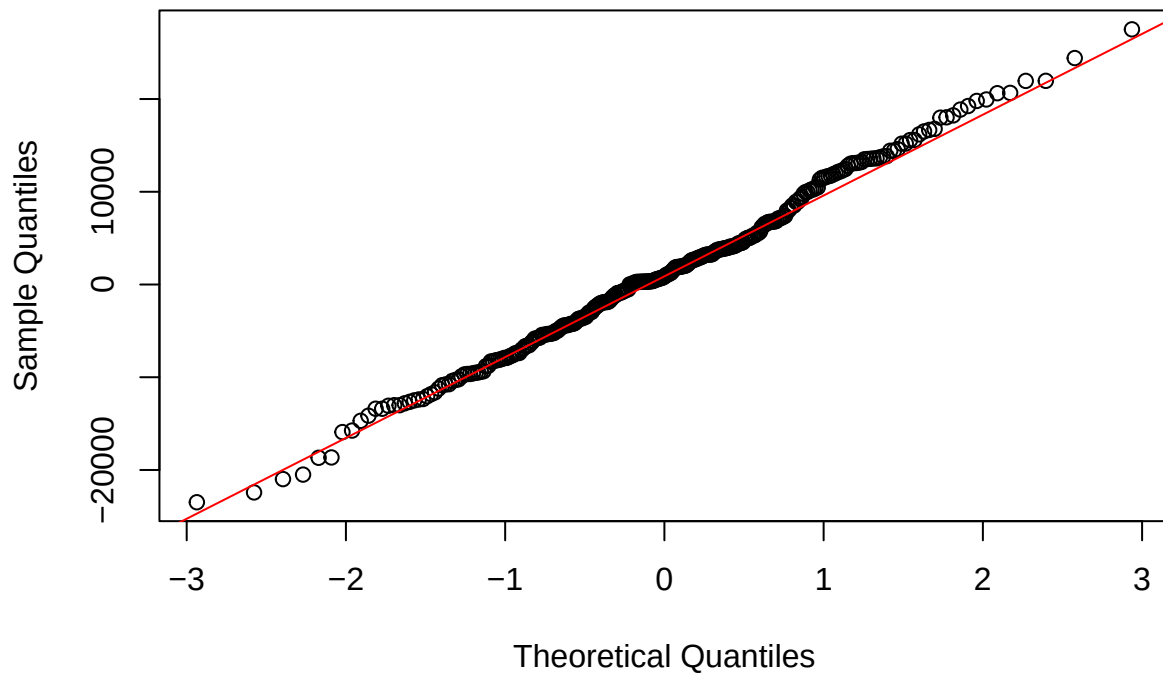


```
Box.test(residuals(model4), lag = 20, type = "Ljung-Box")
```

```
##  
## Box-Ljung test  
##  
## data: residuals(model4)  
## X-squared = 24.551, df = 20, p-value = 0.2191
```

```
qqnorm(residuals(model4), main = "ARIMA(1,0,1)x(0,1,2)12 QQ Plot"); qqline(residuals(model4))
```

ARIMA(1,0,1)x(0,1,2)12 QQ Plot



```
shapiro.test(residuals(model4))
```

```
##  
##  Shapiro-Wilk normality test  
##  
## data:  residuals(model4)  
## W = 0.99544, p-value = 0.5248
```

Summary of tests:

The ACF graph shows only one lag slightly breaking the critical boundaries, indicating the residuals are independent. This result is confirmed by the Ljung-Box test which has a large p-value of 0.2191, so I fail to reject the null hypothesis that the residuals are not autocorrelated. The QQ plot shows the residuals are approximately normally distributed as the values generally remain close to the line. This result is confirmed by the Shapiro-Wilk test which has a large p-value of 0.5248, so I fail to reject the null hypothesis of normality.

Summary of All Models

Model	AIC	Ljung-Box p-value	Shapiro-Wilk p-value
$ARIMA(0, 0, 1) \times (0, 1, 1)_{12}$	6194.13	2.417e-07	0.2611
$ARIMA(1, 0, 1) \times (0, 1, 1)_{12}$	6127.80	0.1306	0.7578
$ARIMA(0, 0, 1) \times (0, 1, 2)_{12}$	6190.28	1.391e-06	0.7743**
$ARIMA(1, 0, 1) \times (0, 1, 2)_{12}$	6126.42**	0.2191**	0.5248

** represents best score

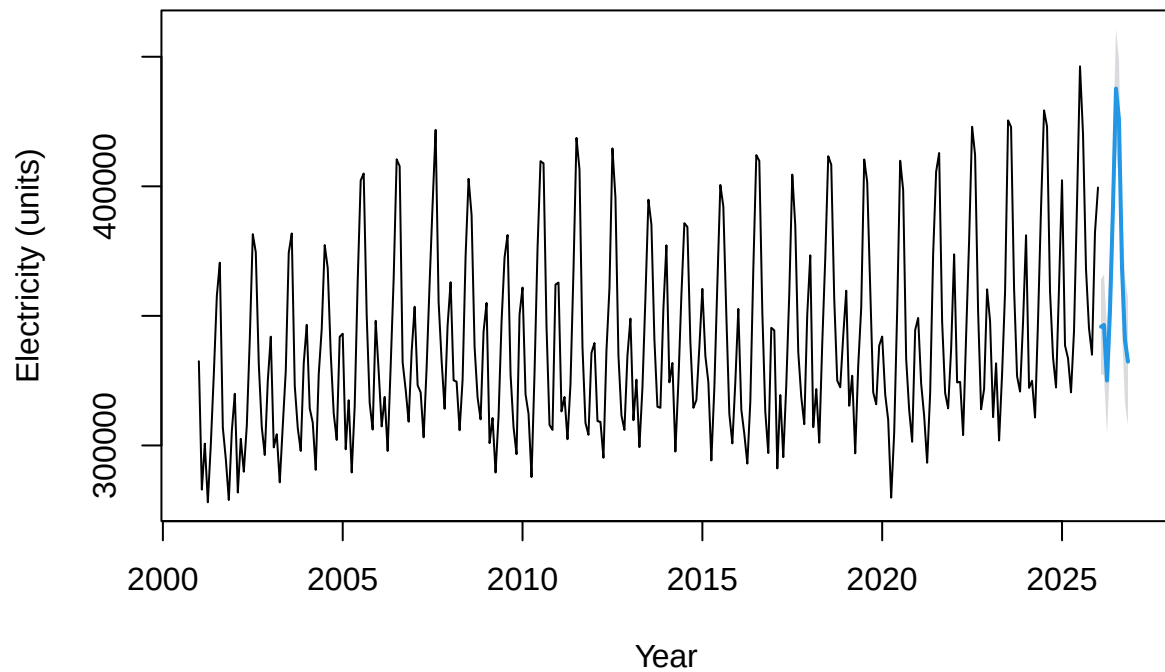
Based on all diagnostic tests, the $ARIMA(1,0,1)x(0,1,2)_{12}$ is the best fitting out of the four based on the Ljung-Box test AIC, so I will use it for forecasting.

1.5 Forecasting

The following two graphs show a 10 month forecast from the end of the data. The first shows the entire range of data from 2001, the second restricts the range to begin in 2024.

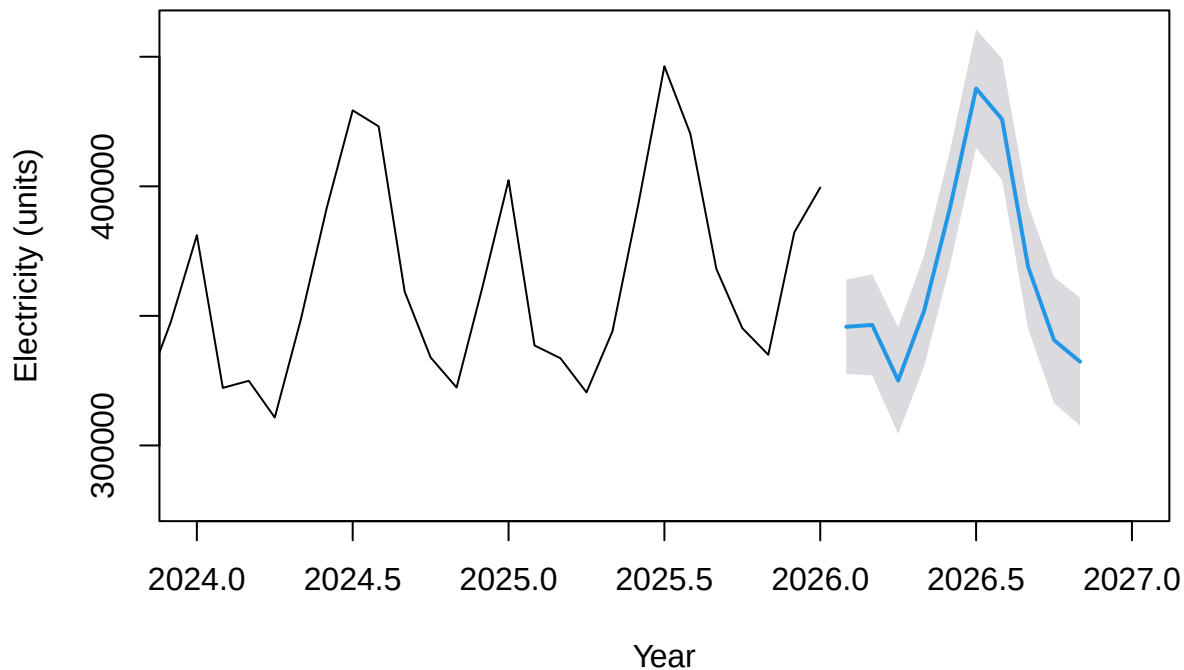
```
plot(forecast(model4, h = 10, level = 95),  
     main = "10-Months Ahead Forecast - ARIMA(1,0,1)x(0,1,2)12 (2001-2027)",  
     xlab = "Year",  
     ylab = "Electricity (units)",  
     xlim = c(2001, 2027))
```

10-Months Ahead Forecast – ARIMA(1,0,1)x(0,1,2)12 (2001–2027)



```
plot(forecast(model4, h = 10, level = 95),  
     main = "10-Months Ahead Forecast - ARIMA(1,0,1)x(0,1,2)12 (2024-2027)",  
     xlab = "Year",  
     ylab = "Electricity (units)",  
     xlim = c(2024, 2027))
```


10-Months Ahead Forecast – ARIMA(1,0,1)x(0,1,2)12 (2024–2027)



The model predicts a dip in electricity generation in the winter of 2026. The generation is then predicted to rise to a peak in summer 2026, and then steadily decline into the fall. The prediction follows the well defined seasonal trend of the rest of the data set. The prediction intervals of the model are fairly small, indicating the model has high confidence in this prediction. The values of the 10 predictions, as well as their 95% prediction interval values are given in the table below.

```
print(forecast(model4, h = 10, level = 95))
```

##	Point Forecast	Lo 95	Hi 95
## Feb 2026	345793.0	327568.6	364017.4
## Mar 2026	346511.2	327016.3	366006.0
## Apr 2026	325026.0	304474.4	345577.5
## May 2026	351989.8	330548.4	373431.3
## Jun 2026	391903.8	369706.2	414101.4

## Jul 2026	437736.6	414892.1	460581.1
## Aug 2026	425865.1	402464.1	449266.0
## Sep 2026	369012.9	345131.5	392894.4
## Oct 2026	340663.0	316365.0	364960.9
## Nov 2026	332344.6	307684.8	357004.3

2 Part II: Cryptocurrency Return Data

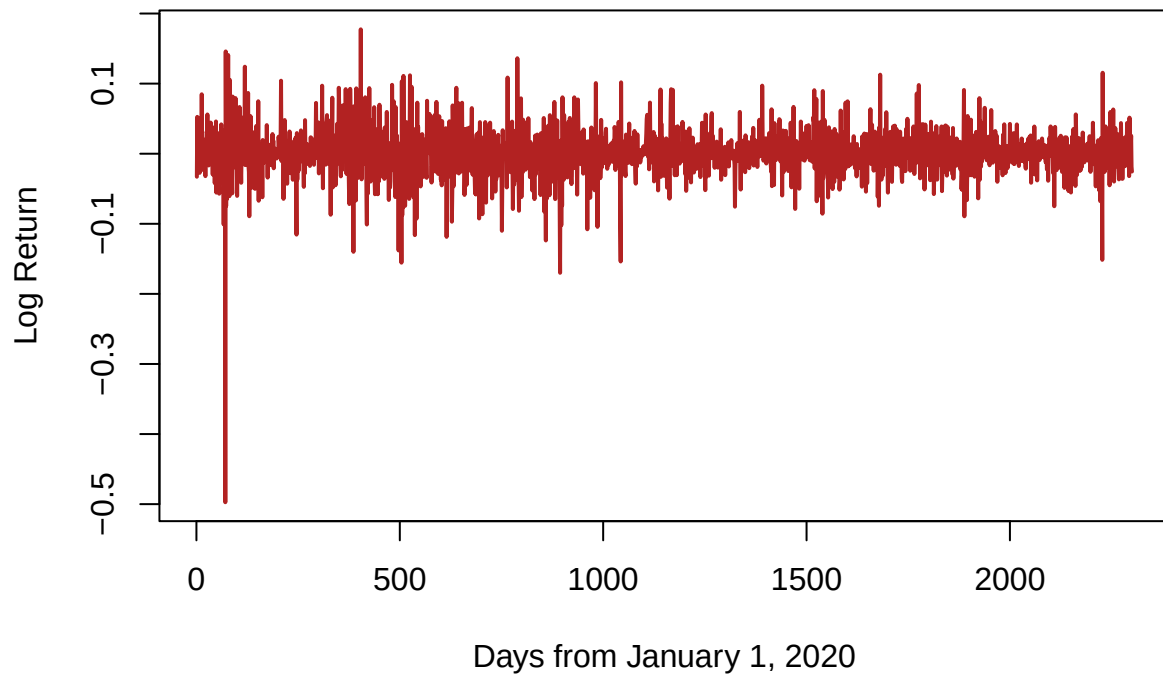
In this section, I will examine the volatility of cryptocurrency return data from 2020-2026.

2.1 Exploratory Data Analysis

```
crypto_returns <-
  read_excel("crypto_log_returns.xlsx")
crypto_returns$Log_return <- as.numeric(crypto_returns$Log_return)

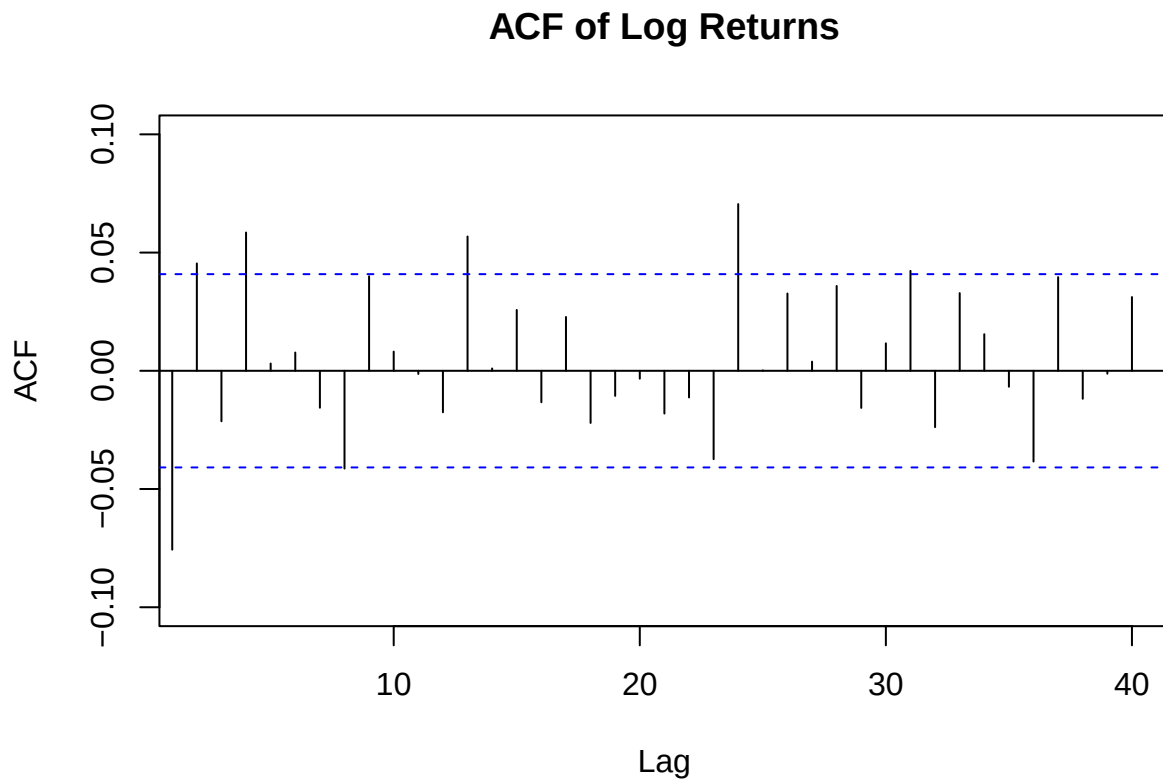
plot(crypto_returns$Log_return,
     main = "Daily Cryptocurrency Log Returns",
     xlab = "Days from January 1, 2020",
     ylab = "Log Return",
     type = "l",
     col = "firebrick",
     lwd = 2)
```

Daily Cryptocurrency Log Returns



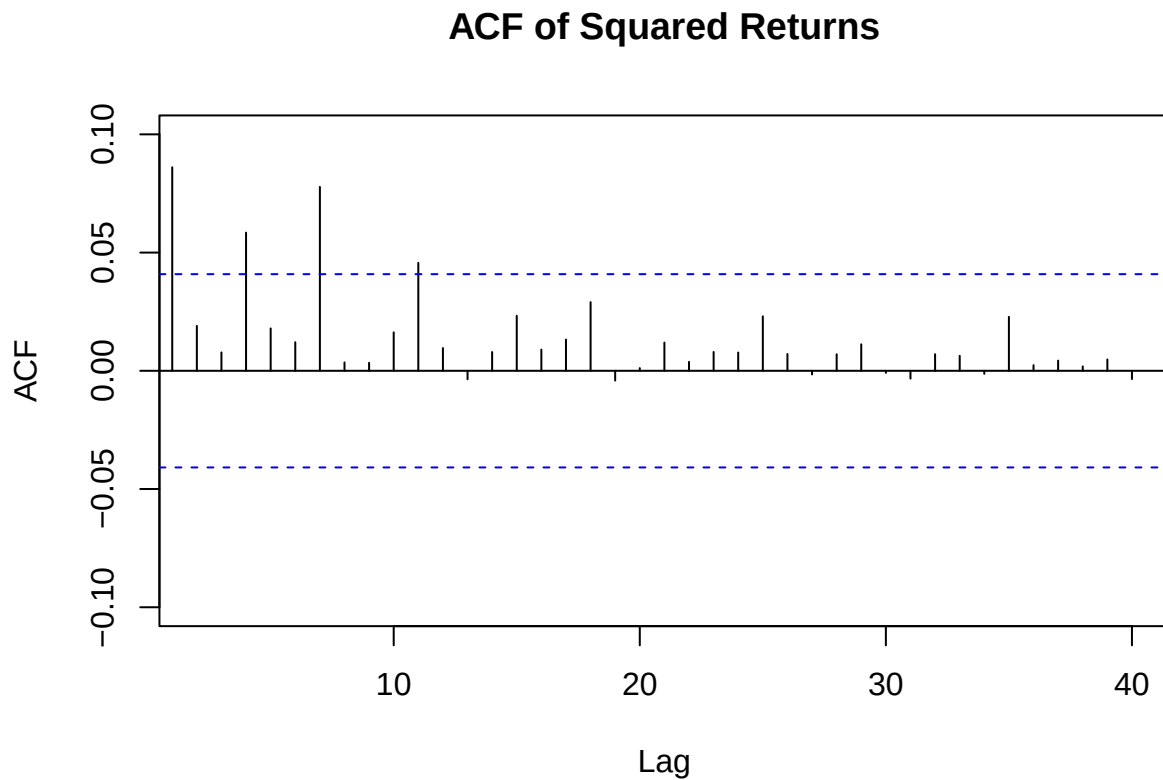
The plot of daily log returns appears to show volatility clustering. The most significant happens at the beginning, at the start of the COVID pandemic. It proceeds into a period of seemingly high volatility before settling down around 1,100 days after January 1, 2020. More recently, there is another spike in volatility that settled soon after.

```
acf(crypto_returns$Log_return,  
    main = "ACF of Log Returns",  
    lag.max = 40,  
    xlim = c(2, 40),  
    ylim = c(-0.1, 0.1))
```



The ACF of log returns shows little significant autocorrelation across all lags, with only a few lags slightly exceeding the critical boundaries. This suggests the returns are largely unpredictable in terms of direction and that little mean structure exists in the series.

```
acf(crypto_returns$Log_return^2,  
    main = "ACF of Squared Returns",  
    lag.max = 40,  
    xlim = c(2, 40),  
    ylim = c(-0.1, 0.1))
```



The ACF of squared returns shows several significant spikes at early lags, particularly at lags 1, 4, 5, 7 and 12, before trailing off. This pattern, along with the lack of autocorrelation in the ACF of returns, shows that the squared returns are not independently and identically distributed. This provides evidence of volatility clustering and the trailing off behavior suggests a GARCH model would fit this data well.

2.2 Arch Effect Test

```
library(TSA)
```

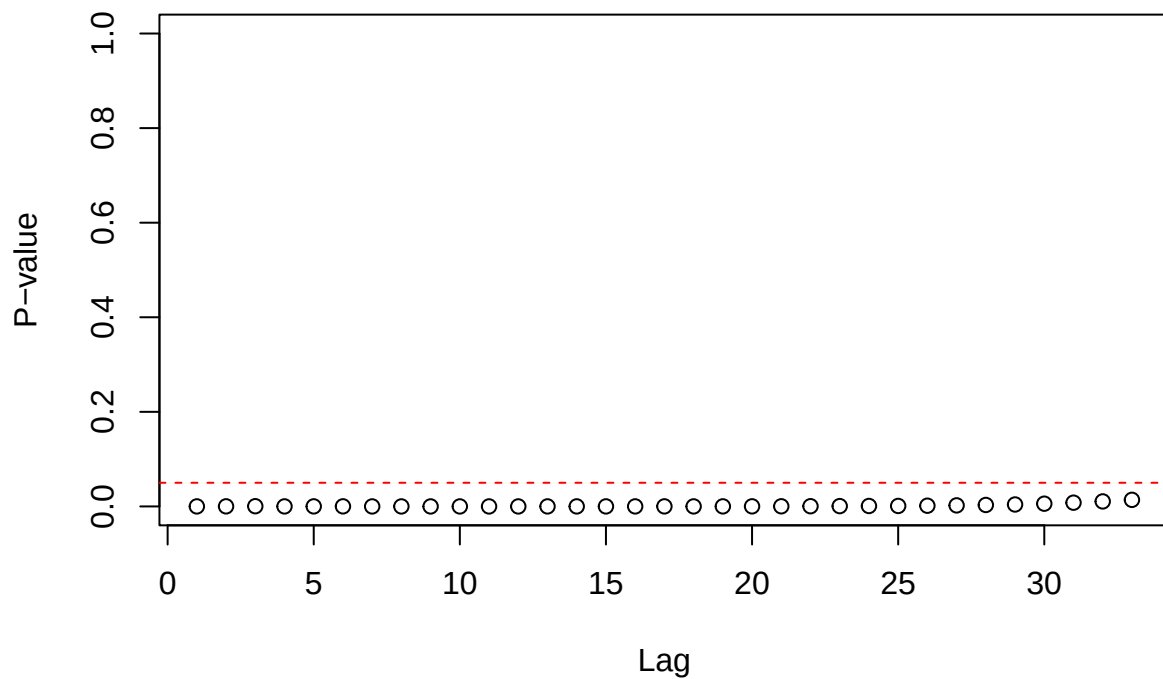
```
## Registered S3 methods overwritten by 'TSA':  
##   method      from  
##   fitted.Arima forecast  
##   plot.Arima  forecast
```

```
##
## Attaching package: 'TSA'

## The following objects are masked from 'package:stats':
##
##   acf, arima

## The following object is masked from 'package:utils':
##
##   tar
```

```
McLeod.Li.test(y = crypto_returns$Log_return)$p.values
```



```
## [1] 3.606213e-05 1.297843e-04 4.318412e-04 3.289831e-05 6.646705e-05
## [6] 1.450086e-04 8.211774e-07 2.087277e-06 4.952658e-06 8.703135e-06
## [11] 2.681795e-06 5.310610e-06 1.084496e-05 2.035510e-05 2.446358e-05
```

```
## [16] 4.275589e-05 6.719301e-05 6.013065e-05 1.036249e-04 1.760492e-04
## [21] 2.622252e-04 4.210325e-04 6.383101e-04 9.515089e-04 1.011443e-03
## [26] 1.476153e-03 2.184434e-03 3.082672e-03 4.087364e-03 5.762437e-03
## [31] 7.946061e-03 1.056471e-02 1.392776e-02
```

The Mcleod_Li test produced significant p-values at every lag, so I reject the null hypothesis and conclude there are ARCH effects present in the cryptocurrency return data.

2.3 Model Fitting

The trailing behavior of the ACF of squared returns suggested a GARCH model would be appropriate, so I will test a GARCH(1,1) model as well as an ARCH(1) model for comparison.

2.3.1 ARCH(1)

```
arch_model <- garch(crypto_returns$Log_return, order = c(1,0))
```

```
##
## ***** ESTIMATION WITH ANALYTICAL GRADIENT *****
##
##
##      I      INITIAL X(I)      D(I)
##
##      1      1.016968e-03      1.000e+00
##      2      5.000000e-02      1.000e+00
##
##      IT      NF      F      RELDF      PRELDF      RELDX      STPPAR      D*STEP      NPRELDF
##      0      1 -6.712e+03
##      1      9 -6.712e+03 2.01e-08 4.29e-08 4.9e-06 1.2e+09 4.9e-07 2.54e+01
##      2      22 -6.712e+03 2.71e-16 1.02e-13 2.0e-10 1.6e+03 2.0e-11 1.09e-07
##      3      25 -6.712e+03 6.78e-16 1.15e-14 2.2e-11 1.4e+04 2.2e-12 1.09e-07
##      4      29 -6.712e+03 1.36e-16 3.33e-17 6.4e-14 4.9e+06 6.4e-15 1.09e-07
```

```
##      5      31 -6.712e+03 -2.57e-15  6.66e-18  1.3e-14  2.5e+07  1.3e-15  1.09e-07
##
## ***** FALSE CONVERGENCE *****
##
## FUNCTION      -6.712066e+03      RELDX          1.276e-14
## FUNC. EVALS      31          GRAD. EVALS      5
## PRELDF          6.658e-18      NPRELDF        1.090e-07
##
##      I      FINAL X(I)      D(I)      G(I)
##
##      1      1.017462e-03      1.000e+00      -3.503e+01
##      2      5.000000e-02      1.000e+00      6.959e-04

## Warning in garch(crypto_returns$Log_return, order = c(1, 0)): singular
## information
```

```
summary(arch_model)
```

```
##
## Call:
## garch(x = crypto_returns$Log_return, order = c(1, 0))
##
## Model:
## GARCH(1,0)
##
## Residuals:
##      Min      1Q   Median      3Q      Max
## -15.19504 -0.39177  0.01387  0.45712  5.42145
##
## Coefficient(s):
##      Estimate Std. Error t value Pr(>|t|)
## a0  0.001017      NA      NA      NA
## b1  0.050000      NA      NA      NA
##
## Diagnostic Tests:
```



```
## Jarque Bera Test
##
## data: Residuals
## X-squared = 65282, df = 2, p-value < 2.2e-16
##
##
## Box-Ljung test
##
## data: Squared.Residuals
## X-squared = 17.061, df = 1, p-value = 3.62e-05
```

Model Form:

$$\sigma_{t|t-1}^2 = 0.001017 + 0.0500r_{t-1}^2$$

The estimated parameters are a0: 0.001017 and b1: 0.0500

The a0 term represents the baseline variance always present in the model. It states this baseline variance of the log cryptocurrency returns is 0.001017. The b1 term represents the proportion of the previous days squared return the model adds to the baseline. In this case, the model states that only 0.05, or 5 percent, of the previous days squared return impacts today's variability. This low percentage indicates that volatility in the crypto market is short lived and easily resolved, which doesn't exactly line up with the graph of crypto returns.

2.3.2 GARCH(1,1)

```
garch_model <- garch(crypto_returns$Log_return, order = c(1,1))
```

```
##
## ***** ESTIMATION WITH ANALYTICAL GRADIENT *****
##
##
```

```

##      I      INITIAL X(I)      D(I)
##
##      1      9.634438e-04      1.000e+00
##      2      5.000000e-02      1.000e+00
##      3      5.000000e-02      1.000e+00
##
##      IT      NF      F      RELDF      PRELDF      RELDX      STPPAR      D*STEP      NPRELDF
##      0      1 -6.735e+03
##      1      7 -6.735e+03  2.40e-05  4.31e-05  1.5e-04  1.3e+09  1.5e-05  2.72e+04
##      2      8 -6.735e+03  5.65e-07  5.64e-07  1.5e-04  2.0e+00  1.5e-05  1.37e+00
##      3      14 -6.738e+03  3.80e-04  6.25e-04  2.4e-01  2.0e+00  3.1e-02  1.37e+00
##      4      18 -6.769e+03  4.62e-03  5.95e-04  8.3e-01  9.5e-01  5.0e-01  5.14e-03
##      5      20 -6.774e+03  7.18e-04  2.86e-03  8.3e-02  2.0e+00  9.9e-02  2.35e+01
##      6      21 -6.815e+03  6.01e-03  5.80e-03  7.1e-02  2.0e+00  9.9e-02  2.02e+01
##      7      23 -6.852e+03  5.44e-03  4.15e-03  6.2e-02  2.0e+00  9.9e-02  8.48e+01
##      8      25 -6.860e+03  1.15e-03  1.61e-03  1.2e-02  2.0e+00  2.0e-02  4.98e+03
##      9      26 -6.865e+03  7.52e-04  1.17e-03  1.1e-02  2.0e+00  2.0e-02  1.32e+03
##      10     28 -6.865e+03  2.97e-05  2.26e-04  2.0e-03  2.0e+00  3.6e-03  2.31e+02
##      11     29 -6.866e+03  8.96e-05  1.42e-04  2.0e-03  2.0e+00  3.6e-03  1.67e+01
##      12     30 -6.866e+03  7.44e-07  1.53e-05  2.0e-03  2.0e+00  3.6e-03  9.39e-01
##      13     35 -6.866e+03  1.02e-06  1.87e-06  1.6e-07  9.2e+00  2.9e-07  1.05e+00
##      14     36 -6.866e+03  1.76e-09  2.34e-09  1.6e-07  2.0e+00  2.9e-07  2.59e-01
##      15     43 -6.866e+03  7.26e-06  1.48e-05  1.3e-03  2.0e+00  2.4e-03  2.58e-01
##      16     46 -6.868e+03  3.76e-04  5.35e-04  1.4e-02  7.1e-01  3.4e-02  7.95e-04
##      17     47 -6.869e+03  6.26e-06  3.95e-05  4.1e-03  0.0e+00  8.8e-03  3.95e-05
##      18     48 -6.869e+03  2.46e-05  2.37e-05  1.4e-03  0.0e+00  2.9e-03  2.37e-05
##      19     49 -6.869e+03  2.33e-07  1.91e-07  3.7e-04  0.0e+00  7.7e-04  1.91e-07
##      20     50 -6.869e+03  7.07e-09  3.98e-09  4.7e-05  0.0e+00  8.8e-05  3.98e-09
##      21     51 -6.869e+03  2.33e-10  7.23e-11  4.0e-06  0.0e+00  6.8e-06  7.23e-11
##      22     52 -6.869e+03 -2.41e-12  6.73e-13  8.3e-07  0.0e+00  1.7e-06  6.73e-13
##
## ***** RELATIVE FUNCTION CONVERGENCE *****
##
##      FUNCTION      -6.868684e+03      RELDX      8.312e-07

```

```
## FUNC. EVALS      52      GRAD. EVALS      22
## PRELDF          6.731e-13    NPRELDF      6.731e-13
##
##      I      FINAL X(I)      D(I)      G(I)
##
##      1      3.582063e-05    1.000e+00    -1.545e+00
##      2      1.214311e-01    1.000e+00    -9.507e-03
##      3      8.611277e-01    1.000e+00    -4.789e-03
```

```
summary(garch_model)
```

```
##
## Call:
## garch(x = crypto_returns$Log_return, order = c(1, 1))
##
## Model:
## GARCH(1,1)
##
## Residuals:
##      Min      1Q   Median      3Q      Max
## -13.37600  -0.41416   0.01589   0.49717   5.01131
##
## Coefficient(s):
##      Estimate Std. Error t value Pr(>|t|)
## a0 3.582e-05   4.257e-06   8.415  <2e-16 ***
## a1 1.214e-01   7.029e-03  17.275  <2e-16 ***
## b1 8.611e-01   9.431e-03  91.306  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Diagnostic Tests:
## Jarque Bera Test
##
## data:  Residuals
## X-squared = 26423, df = 2, p-value < 2.2e-16
```

```
##
##
## Box-Ljung test
##
## data: Squared.Residuals
## X-squared = 0.0089184, df = 1, p-value = 0.9248
```

Model Form:

$$\sigma_{t|t-1}^2 = 0.00003582 + 0.8611r_{t-1}^2 + 0.1214\sigma_{t-1|t-2}^2$$

The estimated parameters are a0: 0.00003582, a1: 0.1214, and b1: 0.8611.

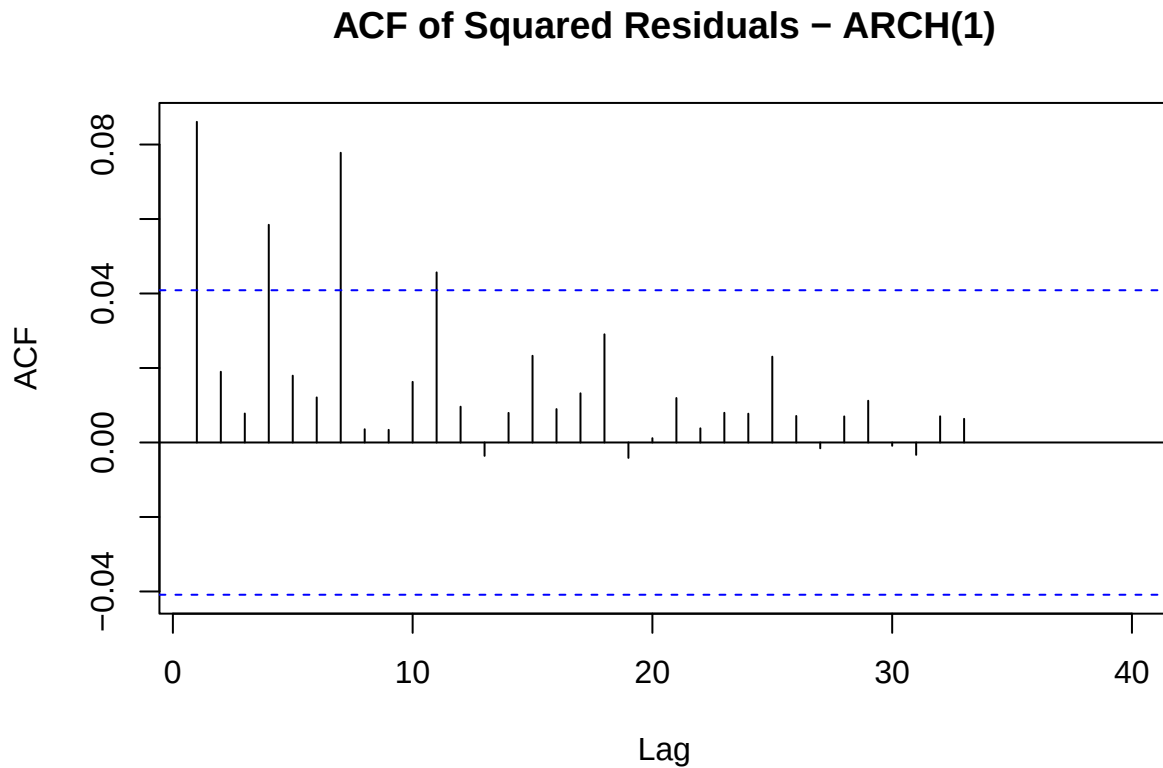
Once again, the a0 term represents the baseline variance of the model. This means that a variance of 0.00003582 will always be present no matter what the other values in the model. The b1 term is the ARCH term which represents the percentage of the previous days squared returns that affects this days variance. In this model it is 0.8611, or 86.11 percent. Finally, the a1 term is the GARCH term which represents the percentage of the previous days conditional variance included in this days variance. In this model it is 0.1214, or 12.14 percent. The 86.11 percent and 12.14 percent indicate that in this model, the volatility of the crypto market is persistent and takes many days to dissipate. This seems to be closer to what was observed in the original graph of the crypto returns.

2.4 Diagnostic Checking

I will now run diagnostic tests on both models to assess which one is a better fit for the cryptocurrency return data.

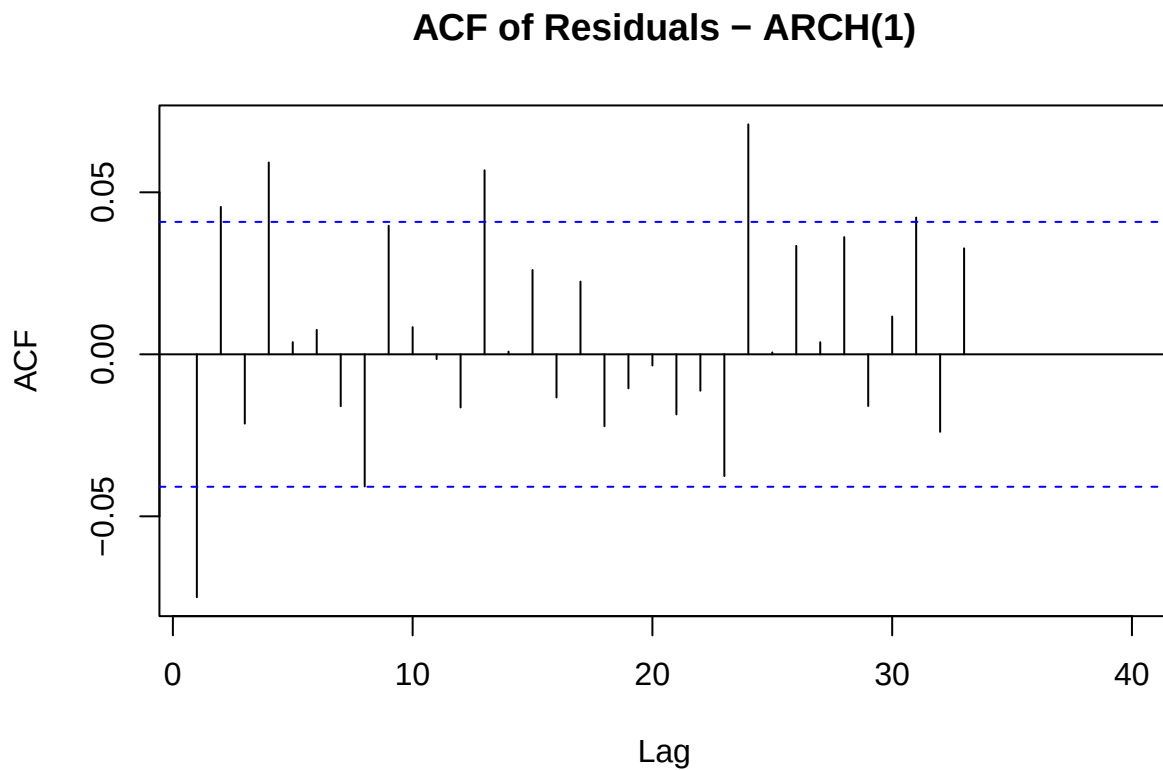
2.4.1 ARCH(1)

```
acf(residuals(arch_model)^2, na.action = na.omit,
    main = "ACF of Squared Residuals - ARCH(1)",
    xlim = c(1, 40))
```



The ACF of squared residuals for the ARCH(1) model shows several large spikes outside the critical boundaries, particularly at early lags. This indicates that ARCH effects remain in the residuals and the model has not adequately captured the volatility clustering present in the data.

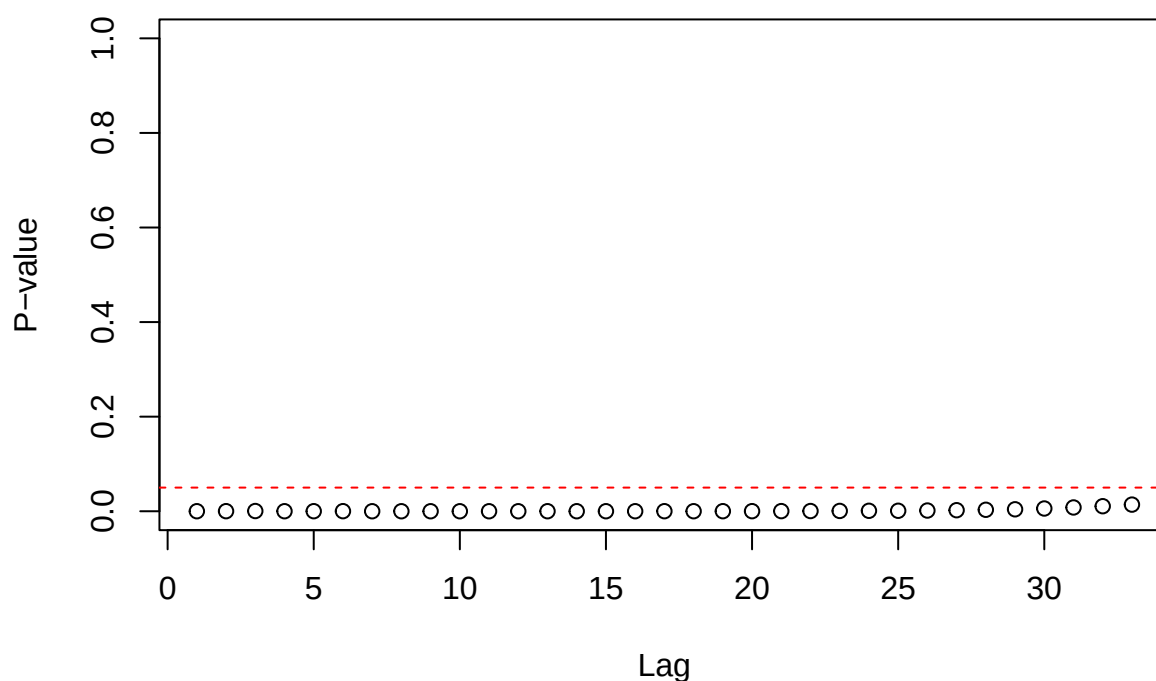
```
acf(residuals(arch_model), na.action = na.omit,
    main = "ACF of Residuals - ARCH(1)",
    xlim = c(1, 40))
```



he ACF of residuals for the ARCH(1) model shows many large spikes throughout the lags, indicating significant autocorrelation remains in the residuals which violates the assumption of uncorrelated residuals.

```
McLeod.Li.test(y = na.omit(residuals(arch_model)),  
               main = "McLeod-Li Test - ARCH(1) Residuals")
```

McLeod-Li Test – ARCH(1) Residuals



The McLeod_Li Test shows low p-values for all lags. This indicates the ARCH(1) model is not capturing the volatility clustering and large ARCH effects still remain in the residuals. This is another indication that the ARCH(1) model is not appropriate for the data.

```
jarque.bera.test(na.omit(residuals(arch_model)))
```

```
##
```

```
## Jarque Bera Test
```

```
##
```

```
## data: na.omit(residuals(arch_model))
```

```
## X-squared = 65282, df = 2, p-value < 2.2e-16
```

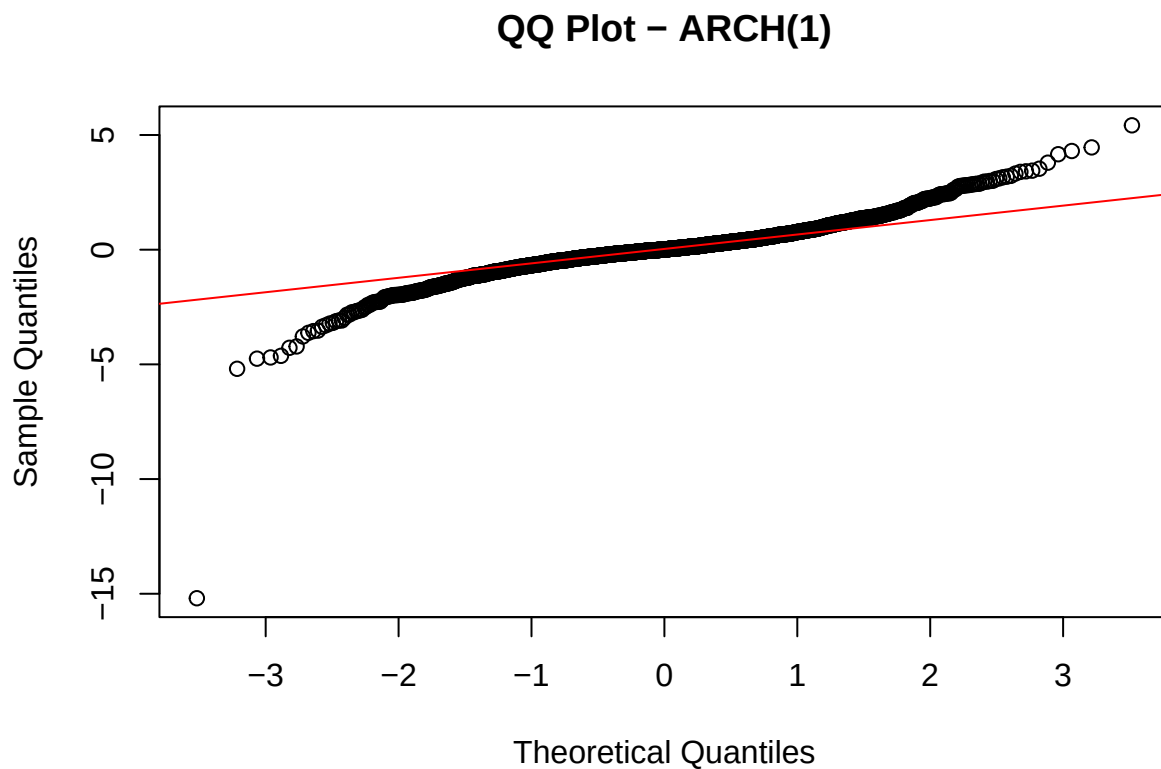
The Jarque-Bera test returns a p-value less than $2.2e-16$, rejecting the null hypothesis of normality. The ARCH(1) residuals are not normally distributed, violating an assumption of normality.

```
shapiro.test(na.omit(residuals(arch_model)))
```

```
##  
##  Shapiro-Wilk normality test  
##  
## data:  na.omit(residuals(arch_model))  
## W = 0.88779, p-value < 2.2e-16
```

The Shapiro-Wilk test is more evidence to this end. The low p-value shows the exact same conclusion of non-normal residuals.

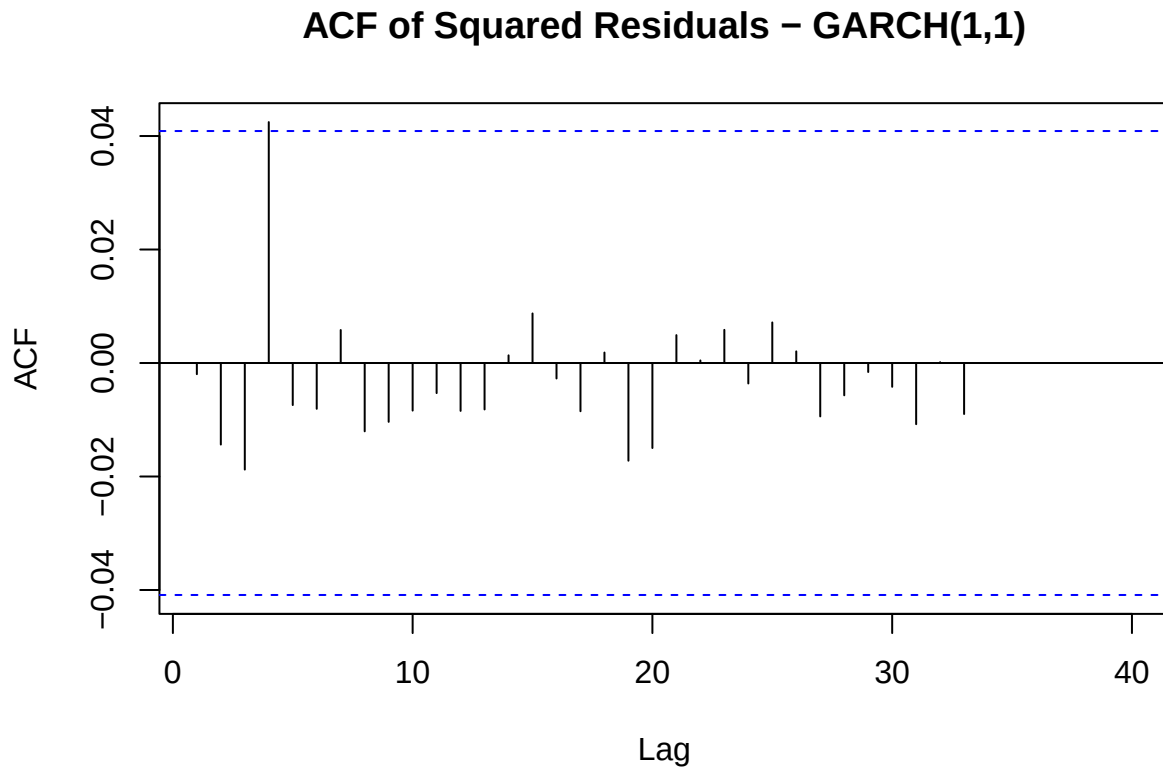
```
qqnorm(na.omit(residuals(arch_model)), main = "QQ Plot - ARCH(1)")  
qqline(na.omit(residuals(arch_model)), col = "red")
```



The QQ plot verifies the previous tests visually. The heavy tails of the residual distribution are shown here, showing they're not normally distributed.

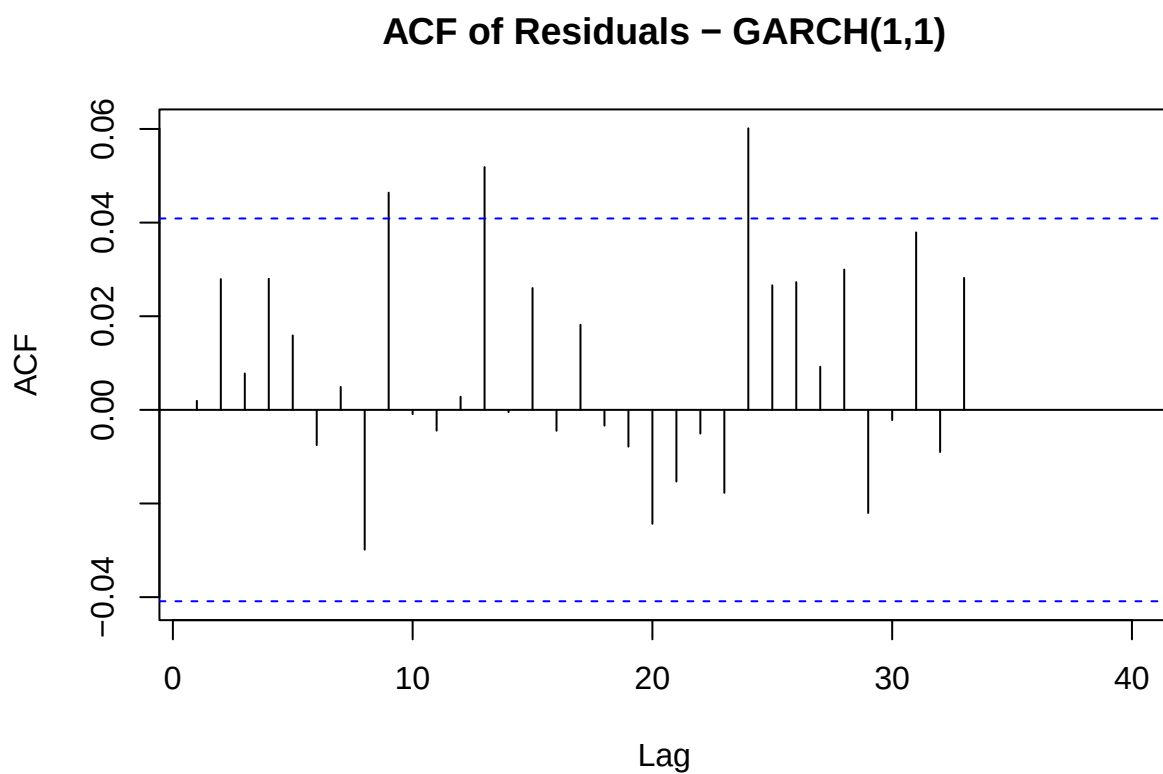
2.4.2 GARCH(1,1)

```
acf(residuals(garch_model)^2, na.action = na.omit,  
    main = "ACF of Squared Residuals - GARCH(1,1)",  
    xlim = c(1, 40))
```



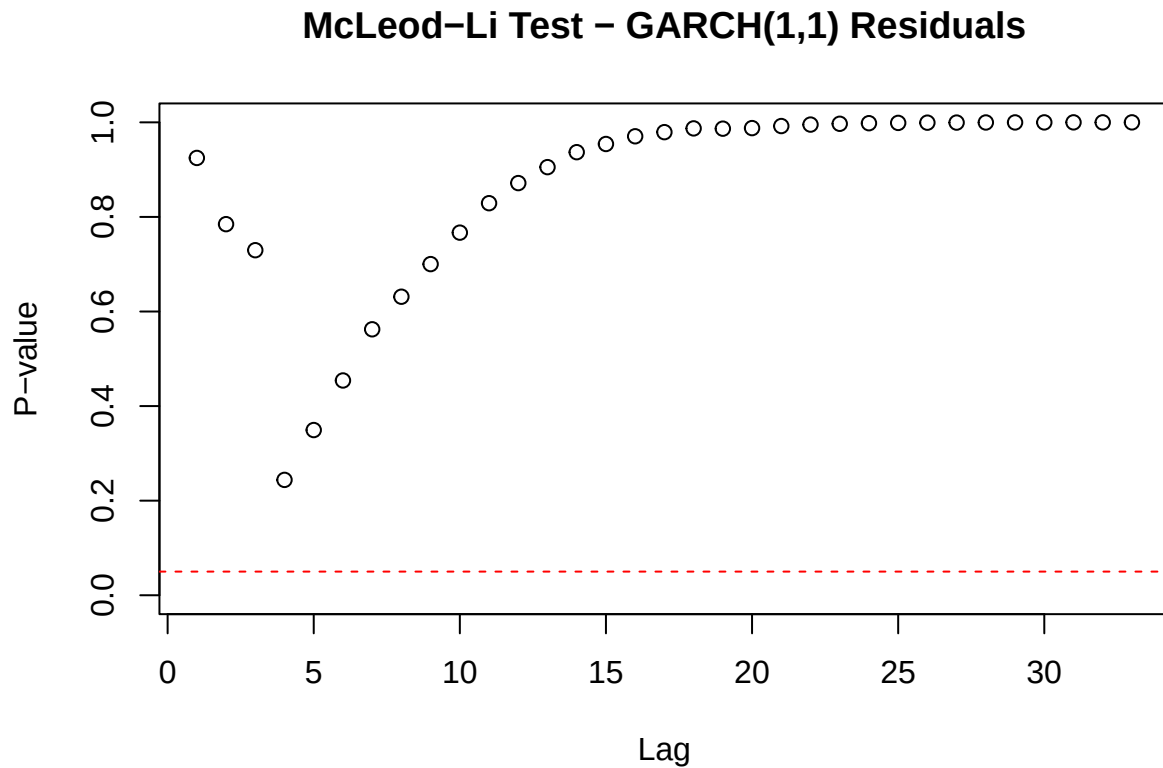
The ACF of squared residuals for the GARCH(1,1) model shows almost no large spikes, with only one lag slightly touching the critical boundary. This shows that the GARCH(1,1) model has successfully negated the ARCH effects on the residuals.

```
acf(residuals(garch_model), na.action = na.omit,  
    main = "ACF of Residuals - GARCH(1,1)",  
    xlim = c(1, 40))
```



The ACF of residuals for the GARCH(1,1) model shows some significant spikes, indicating some mean autocorrelation remains. This is expected as GARCH models the variance structure and does not account for any mean structure in the returns.

```
McLeod.Li.test(y = na.omit(residuals(garch_model)),  
               main = "McLeod-Li Test - GARCH(1,1) Residuals")
```



The p-values of the McLeod_Li test are all large, so I fail to reject the null hypothesis that there are no ARCH effects in the residuals. This is another indication the GARCH(1,1) model succeeded at modelling the volatility clusters in the data set.

```
jarque.bera.test(na.omit(residuals(garch_model)))
```

```
##  
##  Jarque Bera Test  
##  
## data:  na.omit(residuals(garch_model))  
## X-squared = 26423, df = 2, p-value < 2.2e-16
```

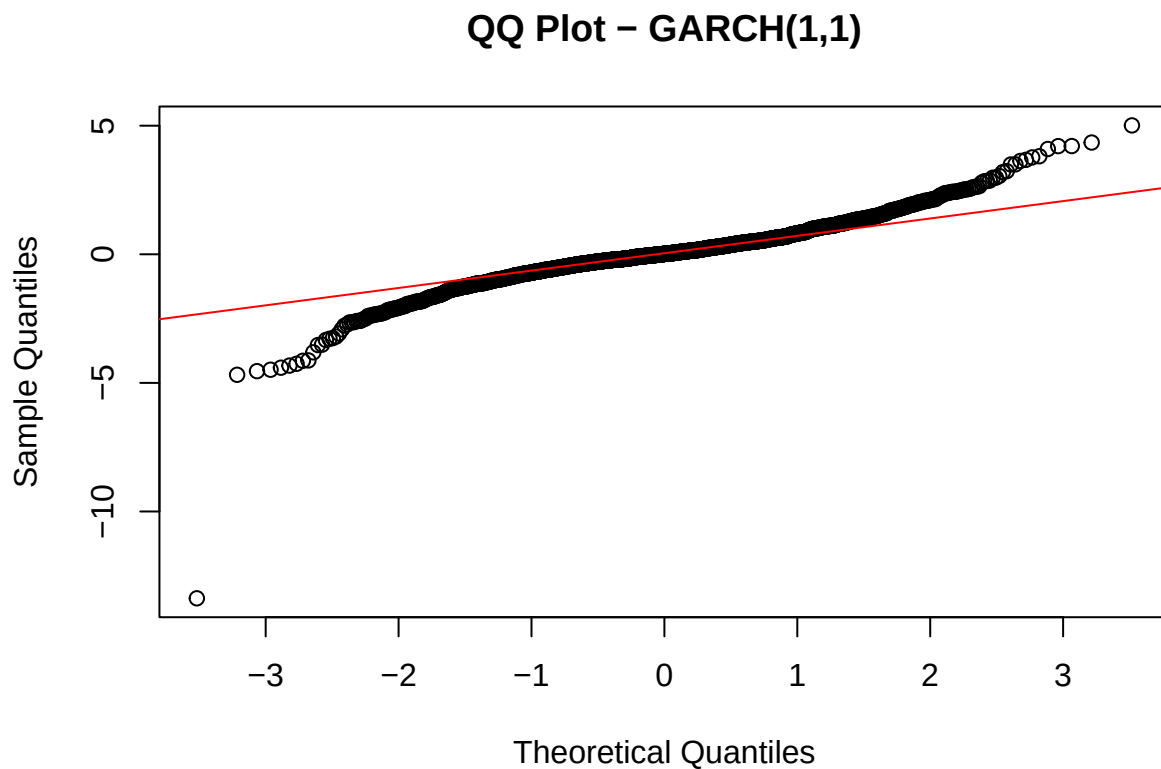
The Jarque-Bera test returns a very low p-value, indicating the residuals are not normally distributed. This is a problem with both the ARCH(1) model and the GARCH(1,1) model.

```
shapiro.test(na.omit(residuals(garch_model)))
```

```
##  
## Shapiro-Wilk normality test  
##  
## data:  na.omit(residuals(garch_model))  
## W = 0.91404, p-value < 2.2e-16
```

The low p-value on the Shapiro-Wilk test is more evidence that the residuals of the GARCH(1,1) model are not normally distributed.

```
qqnorm(na.omit(residuals(garch_model)), main = "QQ Plot - GARCH(1,1)")  
qqline(na.omit(residuals(garch_model)), col = "red")
```



Finally, the QQ plot of the GARCH(1,1) model shows visually the heavy tails of the residuals that makes them not normal.

Summary of tests:

Diagnostic	ARCH(1)	GARCH(1,1)
ACF of Residuals	Significant spikes	Some significant spikes**
ACF of Squared Residuals	Significant spikes	Almost none**
McLeod-Li Test	Low p-values	High p-values**
Jarque-Bera Test	$p < 2.2e-16$	$p < 2.2e-16$
Shapiro-Wilk Test	$p < 2.2e-16$	$p < 2.2e-16$

** represents better performing

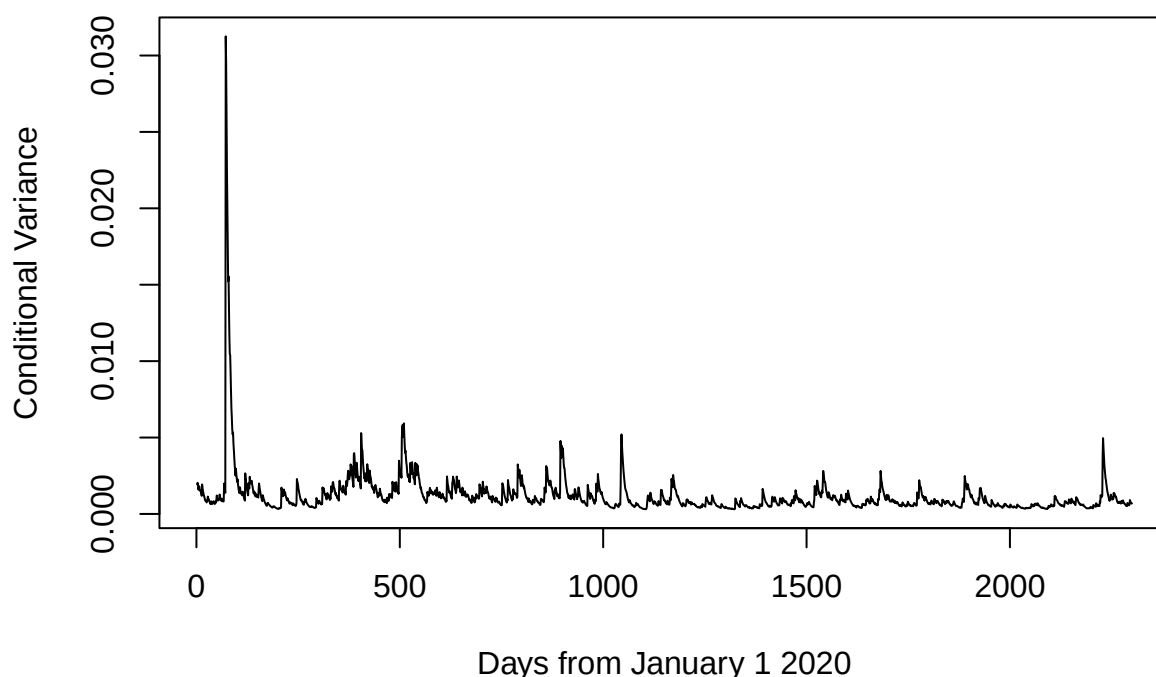
While both models fail the tests for normality of residuals, the GARCH(1,1) model is clearly much more appropriate to model the data and remove the ARCH effects. This comports well with other research done into cryptocurrency returns, which find that models assuming a non-normal residual distribution such as a t-distribution are a better fit for cryptocurrency data (Tran, Molnár, & Sensoy, 2026).

2.5 Volatility Interpretation

To understand what the GARCH(1,1) says about the volatility of cryptocurrency returns, I'll start by plotting the conditional variance of the model.

```
cond_var <- garch_model$fitted.values[,1]^2
plot(cond_var,
     type = "l",
     main = "GARCH(1,1) Estimated Conditional Variance",
     xlab = "Days from January 1 2020",
     ylab = "Conditional Variance")
```

GARCH(1,1) Estimated Conditional Variance



$$\sigma_{t|t-1}^2 = 0.00003582 + 0.8611r_{t-1}^2 + 0.1214\sigma_{t-1|t-2}^2$$

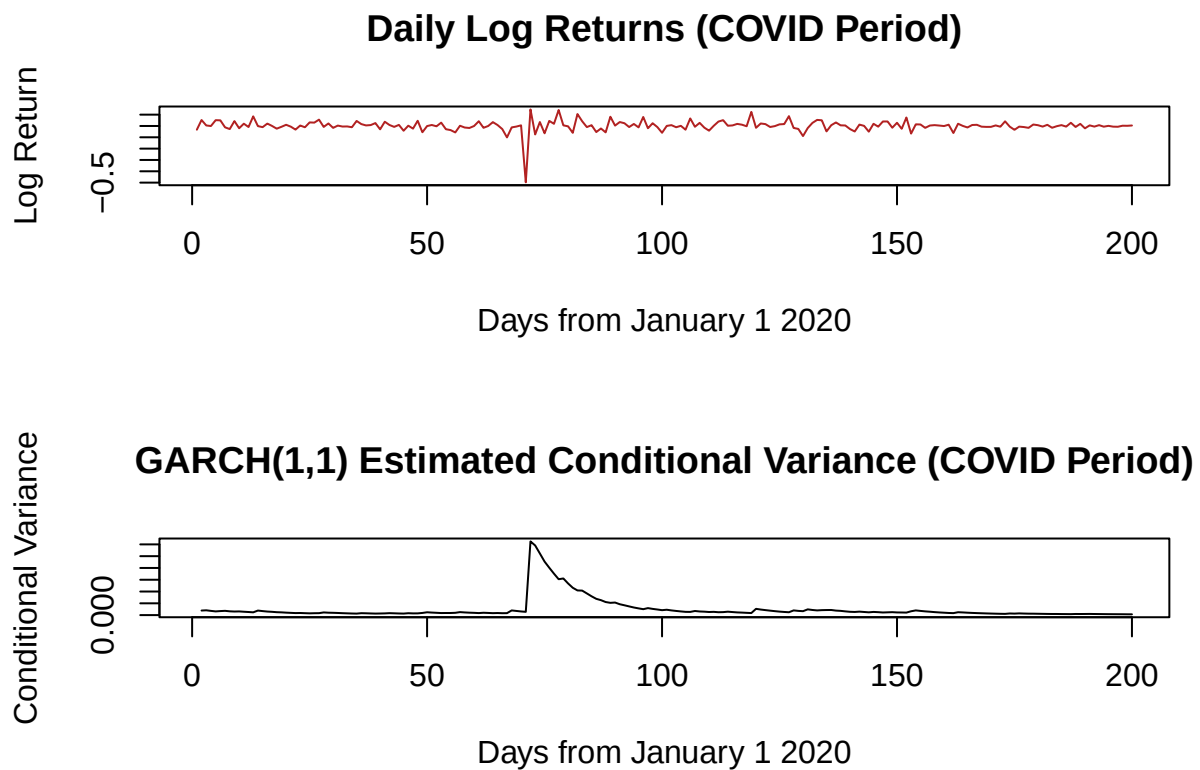
The GARCH(1,1) model measures the conditional variance in the cryptocurrency returns on any day given the squared returns and the conditional variance of the prior day. It does not predict future shocks, but rather it models the way the volatility of returns behaves once a shock happens. Per the model equation, the conditional variance adds 86.11 percent of the previous days squared returns with 12.14 percent of the previous days conditional variance, as well as a baseline variance of 0.00003582. Because these percentages are so high, the model predicts that variance will remain high for many days after a shock, and reduce slowly over time. This can be seen in the above graph, with the high volatility of the COVID shock producing a period of high volatility before it settles into a period of low volatility. According to the model, volatility is persistent but will eventually settle into a low baseline value on a long enough time horizon. The below graph displays this more clearly with an examination of the immediate post-COVID era.

```

par(mfrow = c(2,1))
plot(crypto_returns$Log_return[1:200],
     type = "l",
     main = "Daily Log Returns (COVID Period)",
     xlab = "Days from January 1 2020",
     ylab = "Log Return",
     col = "firebrick")

cond_var <- garch_model$fitted.values[,1]^2
plot(cond_var[1:200],
     type = "l",
     main = "GARCH(1,1) Estimated Conditional Variance (COVID Period)",
     xlab = "Days from January 1 2020",
     ylab = "Conditional Variance")

```



Observing the daily log returns after the COVID shock with the conditional

variance graph in the same time period shows a large spike in variance followed by a slow reduction that takes roughly 35 days to resolve. This shows that volatility is persistent in this GARCH(1,1) model, as a month constitutes a long time for any heightened cryptocurrency return volatility.

3 Sources

Tran, Q. V., Molnár, P., & Sensoy, A. (2026). Unconditional and conditional heavy-tailed distributions for the returns of cryptocurrencies with a novel range exponential GARCH model. *Borsa Istanbul Review*, 26(3), 100803. <https://doi.org/10.1016/j.bir.2026.100803>